



كلية الحاسبات والمعلومات
FACULTY OF COMPUTERS AND INFORMATICS



ZAGAZIG UNIVERSITY
FACULTY OF COMPUTERS AND INFORMATICS
ARTIFICIAL INTELLIGENCE AND DATA SCIENCE PROGRAM

Health Mate

An AI-Assisted Remote Monitoring System
for Chronic Cardiovascular Care

A Graduation Project Documentation

BY

Omar Ashraf Mohamed Kamal *(Team Leader)*

Seif EL-Deen Amr Mohamed

Rofida Mohamed Ahmed Ibrahim

Farida Abdelazim Gharib Mohamed

Under Supervision Of

ASS. Prof. Dr. Amr Abdellatif

Eng. Mohamed Ameen

2025 – 2026

Abstract

Health Mate: An AI-Assisted Remote Monitoring System for Chronic Cardiovascular Care

Omar Ashraf Mohamed Kamal, Seif EL-Deen Amr Mohamed, Rofida Mohamed Ahmed Ibrahim, Farida Abdelazim Gharib Mohamed

Hypertension and related cardiovascular disease require sustained monitoring between clinical visits, yet patients — particularly elderly patients — are inconsistent about manual cuff measurement, and the family members who care for them typically have no visibility into readings unless told directly. **Health Mate** is a remote patient-monitoring system that closes this gap with three connected parts: a custom cuffless blood-pressure sensing unit built from a MAX30102 pulse-oximeter and an AD8232 single-lead ECG front-end; a FastAPI backend that runs a trained deep-learning model to estimate blood pressure from the resulting PPG/ECG signal; and a Flutter mobile application that gives both the patient and a linked caregiver a shared, near-real-time view of the same data.

A second AI subsystem — a symptom/disease triage model backed by a safety-first rule engine — complements the blood-pressure pipeline by letting a patient describe symptoms and receive an urgency-graded response that can independently escalate to a caregiver notification. A third piece of hardware, a smart medicine box, indicates which medication drawer to open on a schedule through an LED and buzzer, paired with cloud-push and on-device reminders.

The blood-pressure model, trained on a MIMIC-derived dataset of more than 650,000 signal windows, was selected from a ten-model comparison. Its best architecture — a plain one-dimensional convolutional network — reaches a diastolic accuracy that clears the AAMI clinical standard (MAE 4.91 mmHg) while systolic accuracy remains just below that bar (MAE 9.08 mmHg); this document reports the result plainly rather than rounding it up. The structured symptom-checker model reaches 85.8% top-1 and 95.5% top-3 accuracy on a held-out synthetic dataset, with its largest weakness concentrated in distinguishing overlapping cardiac subtypes — the project’s own core clinical focus. Throughout, this documentation covers the system’s architecture, the methodology and honest, measured results behind both models, the hardware that feeds them, and the real engineering gaps found during review, so that the system is evaluated on what it actually does rather than on what it was originally intended to do.

Keywords: Cuffless Blood Pressure, PPG/ECG, Deep Learning, Remote Patient Monitoring, Symptom Triage, IoT Health, FastAPI, Flutter, Per-Patient Calibration.

Contents

Abstract	i
1 Introduction	1
1.1 Introduction and Significance	1
1.1.1 Project Objectives	2
1.2 Contribution to Scientific Discovery	3
2 Related Work	5
2.1 Cuffless Blood Pressure Estimation	5
2.2 Rule-Based Clinical Decision Support	6
2.3 Internal Comparative History	6
3 Software Description	7
3.1 Software Architecture	7
3.1.1 Two Architectural Styles, Side by Side	7
3.1.2 Cross-Cutting Backend Pieces	9
3.2 Tools and Technologies Used	9
3.3 Software Functionalities	10
3.4 Functional Requirements	12
3.5 Non-Functional Requirements	13
4 IoT Hardware Subsystems	16
4.1 Blood-Pressure Sensing Unit	16
4.1.1 Hardware	16
4.1.2 Firmware Behavior	18
4.2 Smart Medicine Box	18
4.2.1 Hardware — an Explicit Correction	18
4.2.2 Who Talks to It	19
4.2.3 Reliability Characteristics	19
5 Database Design	21
5.1 Entity-Relationship Diagram	21

5.2	Tables at a Glance	23
5.3	Relationships Worth Calling Out	26
5.4	Business Rules Encoded in the Schema and Domain Layer	26
6	Methodology	27
6.1	Data Collection and Preprocessing	27
6.1.1	Blood-Pressure Dataset	27
6.1.2	Preprocessing Pipeline	28
6.1.3	Feature Engineering	29
6.1.4	Class Imbalance: Sample Weights, Not Downsampling	30
6.1.5	Data Augmentation	31
6.1.6	Symptom-Checker Dataset	31
6.2	Model Development	32
6.2.1	Blood-Pressure Models	32
6.2.2	Symptom-Checker Model	36
6.3	Implementation	36
7	Results and Discussion	37
7.1	Model Performance	37
7.1.1	Blood-Pressure Results	37
7.1.2	Symptom-Checker Results	40
7.2	Interpretation of Results	41
7.3	Calibration Simulation	42
8	Illustrative Examples and Feature Flows	44
8.1	Blood-Pressure Measurement Flow	44
8.1.1	The Post-Inference State Machine	45
8.2	Calibration and Drift Decision Flow	46
8.3	Medication Reminder Flow	47
8.4	Symptom-Triage Flow	49
8.4.1	The Safety Rule Engine	50
8.5	Application Screenshots	51
9	Impact	54
9.1	Target Audience	54
9.2	Problems Solved	55
9.3	Innovative Aspects of the Design	57

9.4	Reliability	57
9.5	Added Value	57
10	Conclusions	59
11	Future Directions	60
11.1	Hardware Sensor Upgrade Path	60
	References	62
A	API Endpoint Reference	64
B	Deployment and Networking	66
C	Documentation Method and Honesty Statement	67
D	Code Snapshots	68
D.1	Hardware BP Submission Gate	68
D.2	Calibration-Aware Measurement State	68
D.3	ABP Model Inference Core	69
D.4	Symptom-Triage Safety Path	69
D.5	Escalate-Only Urgency Rule	70
D.6	Primary-Caregiver Database Constraint	70
D.7	Firmware Signal-Presence Gating	70
D.8	Flutter Symptom-Checker API Call	71

List of Figures

1.1	System context: two human actors, one shared mobile app, one backend that owns every clinical decision, and two purpose-built IoT devices. . . .	2
3.1	Health Mate system architecture. The hardware never makes clinical decisions — every safety-relevant step runs in the backend.	7
3.2	Clean-architecture layering used for the symptom checker v2: the only subsystem restructured this way. Dependencies point inward; infrastructure implements domain interfaces, wired in one dependency-injection module.	8
3.3	Use-case diagram. Patients and caregivers share the calling, linking, alerting, and monitoring use cases; measurement, calibration, medication, and symptom assessment are patient-driven, while “monitor linked patient” is caregiver-specific.	15
4.1	Blood-pressure sensing unit — complete hardware wiring (NodeMCU ESP8266 + MAX30102 PPG + AD8232 ECG), with per-sensor pin tables, electrode placement, and functional notes.	17
4.2	AD8232 three-lead (RA/LA/RL) electrode placement on the torso and limbs, shown in two equivalent placement variants.	17
4.3	Smart medicine box — physical enclosure (six compartments, LED positions, acrylic window) and complete hardware wiring (ESP32 + 74HC595 shift register + six drawer LEDs + active buzzer), with the controller pin map and functional notes.	19
5.1	Complete entity-relationship diagram for the implemented Health Mate backend schema.	22
5.2	Complete database tables reference exported from the implemented backend models.	25
6.1	PPG (MAX30102) preprocessing pipeline: raw ADC counts → bandpass → z-score normalized.	29
6.2	ECG (AD8232) preprocessing pipeline: raw single-ended output → bandpass → z-score normalized.	29
6.3	Correlation of the 21 hand-crafted features with SBP/DBP. PTT and heart rate dominate, but no single feature is strong enough on its own.	30
6.4	BP-category distribution: heavily imbalanced toward the Hypertension and Hypotension extremes.	31
6.5	Inverse-frequency sample weights, clipped to a 5× maximum ratio.	31

6.6	Conceptual two-branch design: a waveform encoder and a dense feature branch, concatenated before a shared prediction head.	33
6.7	Exact Keras layer graph of the best-performing 1D-CNN model (auto-generated by <code>plot_model</code>).	33
6.8	Exact Keras layer graphs of the other three deep-learning architectures. The increasing depth from (a) to (c) reflects the rising parameter counts in Table 6.4 — and, as the results show, that added depth did not translate into better accuracy.	34
6.9	Train-versus-validation MAE gap across models after the regularization course-correction.	35
6.10	Fault-tolerant, resumable training workflow: cache-and-resume at every stage, with per-model isolation.	35
7.1	SBP MAE across all ten models against the AAMI limit (5 mmHg) and the v4 baseline.	38
7.2	DBP MAE across all ten models; the best model clears the AAMI limit for diastolic pressure.	38
7.3	True vs. predicted SBP (top) and DBP (bottom), colored by absolute error.	39
7.4	Bland–Altman agreement for SBP (top) and DBP (bottom): mean bias $\pm$ 1.96 SD limits of agreement.	39
7.5	Cumulative error-threshold curve for the best model against the BHS A/B/C grade bands.	39
7.6	Per-class MAE across the four BP categories: error is not uniform across the pooled AAMI/BHS headline.	39
7.7	Ablation rationale: how the final configuration was reached from the earlier over-parameterized and over-balanced attempts.	40
7.8	Before/after the oracle calibration simulation. The negative result is an artifact of the simulation’s sampling, not a verdict on the deployed service.	43
8.1	Blood-pressure measurement sequence, from the device window upload through model inference, calibration, persistence, and conditional alerting.	45
8.2	The blood-pressure submission state machine, traced from the submission endpoint. The pending-BP branch is what generates the paired samples the calibration fit later depends on.	46
8.3	Per-patient calibration fit and the scheduled drift/staleness job. Drift is flagged on a sustained shift >8 mmHg (SBP) or >5 mmHg (DBP) from baseline, or after 90 days.	47
8.4	Medication reminder sequence: the smart-box activation is fail-soft, so the phone notification is delivered regardless of hardware reachability.	48
8.5	Medication alarm decision tree, including grouping of multiple medications due at the same time into one drawer-activation call.	49

8.6	The two coexisting symptom-checker flows. v1 has no safety layer; v2 runs the ML prediction and the rule engine independently and reflects the more severe of the two.	50
8.7	Guided blood-pressure measurement, from sensor setup to saved history (Arabic UI).	51
8.8	Medication flow: list, add-medicine form, smart-box setup, and the resulting alarm (Arabic UI).	52
8.9	Symptom-triage wizard: category, symptoms, severity, follow-up, and the urgency-graded result (Arabic UI).	52
8.10	WebRTC audio/video calling: outgoing, incoming, active call, and the emergency-call screen (Arabic UI).	53
8.11	Patient home, caregiver patient detail, notifications inbox, and QR-based caregiver linking (Arabic UI).	53

List of Tables

3.1	Technology stack and rationale.	10
3.2	Functional requirements.	12
3.3	Non-functional requirements.	13
3.4	Professional use-case catalogue for the implemented Health Mate system.	14
4.1	The two IoT subsystems at a glance.	16
5.1	The 21 tables of the Health Mate schema.	23
5.2	Key business rules, as encoded in the domain layer.	26
6.1	Final windowed dataset sizes after record-level splitting and quality filtering.	28
6.2	The 21 hand-crafted features fed to the model’s dense side-branch.	30
6.3	Layout of the 232-dimension structured symptom feature vector.	32
6.4	The four deep-learning architectures and their parameter counts.	32
7.1	Final test-set results for all ten blood-pressure models, sorted by SBP MAE. Lower MAE is better; BHS grades run A (best) to D (worst).	37
7.2	The ten lowest per-class top-1 accuracies — almost all cardiac subtypes.	41
7.3	Symptom-checker model comparison on the identical structured dataset.	41
7.4	Oracle calibration simulation: calibration degraded accuracy on this test set.	43
9.1	Real-world problems and the Health Mate mechanism that solves each.	56
A.1	REST API surface, grouped by router.	64

Chapter 1

Introduction

1.1 Introduction and Significance

Hypertension is one of the most prevalent chronic conditions worldwide and the single largest modifiable risk factor for stroke, myocardial infarction, heart failure, and chronic kidney disease. Its danger is compounded by the fact that it is largely asymptomatic: a patient whose pressure has drifted into a dangerous range typically feels nothing until an acute event forces the issue, which means the disease has to be caught by measurement, not by how the patient feels. This is precisely where the standard model of care falls short. A clinic visit every few months produces a single data point — a *snapshot* — when what actually matters clinically is the *trend* across weeks and months. Between those visits, the burden of measurement falls entirely on the patient, and in practice two failure modes recur constantly: patients forget or actively avoid manual cuff readings (correct cuff placement, sitting still for the reading, and then remembering to write the number down are each their own small barrier), and family caregivers — very often adult children managing a parent’s condition from a different household — have no channel to that data at all short of phoning and asking. A caregiver who wants to check in proactively on a bad day currently cannot; they can only react to what they are told, often after the fact.

Health Mate’s central bet is that solving both halves of this problem together — not separately — is what actually changes the outcome. On the measurement side, replacing a manual cuff with a guided, sensor-based flow removes the specific frictions (placement, timing, remembering to log the result) that cause patients to skip readings, without asking the patient to be more disciplined than they already are. On the visibility side, giving a caregiver a live, linked view of the same data removes their dependency on being told, and lets them notice a concerning trend on their own schedule rather than only in a crisis. Neither half is new by itself; a network-connected cuff and a family-sharing health app already exist as separate products. The distinctive engineering claim here is that combining guided cuffless measurement, automatic per-patient calibration, a linked caregiver view, medication support, and a safety-checked symptom triage layer into one coherent system — built and running end to end, not four disconnected demos — is what turns “more consistent monitoring” from a slogan into something a specific patient and a specific caregiver can actually rely on day to day. Figure 1.1 shows the four technical parts that make this possible and how they connect.

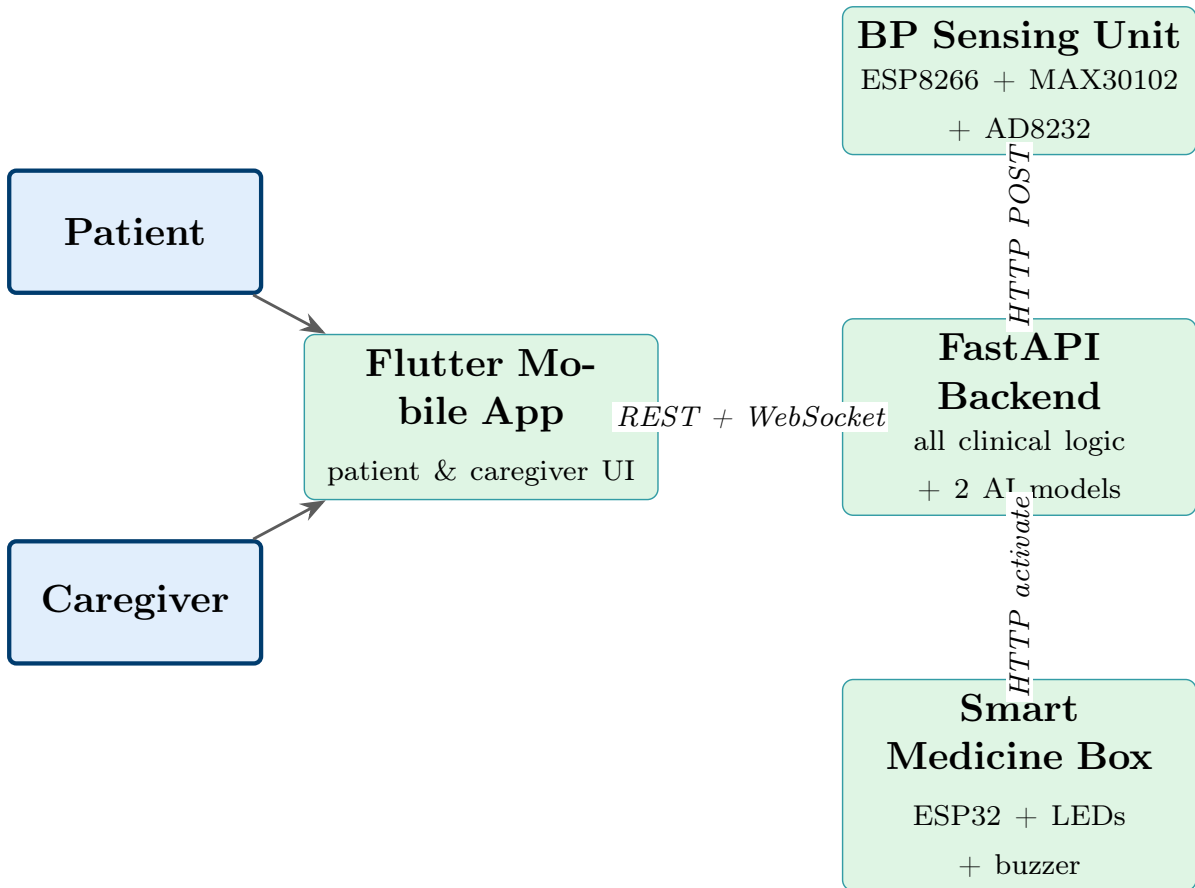


Figure 1.1: System context: two human actors, one shared mobile app, one backend that owns every clinical decision, and two purpose-built IoT devices.

The system was built as a full *vertical slice* rather than a collection of independent components: custom sensor hardware that captures a real physiological signal, a trained model that turns that raw signal into a clinical number, a backend that owns every safety-relevant decision so the hardware itself never has to be trusted with one, and a mobile app that presents the same underlying data appropriately to a patient and to a caregiver from shared code. Building all four layers together, integrated and actually running against each other rather than validated in isolation, is what distinguishes this project’s engineering effort from a single training notebook or a single mobile-app prototype — and it is also why the honest gaps documented throughout this report (Chapter 7, Section 8.5, and the reliability notes in Chapter 10) are named plainly rather than hidden: a system this size, built by a graduation-project team, will have real rough edges, and stating them precisely is more useful to a reader than pretending otherwise.

1.1.1 Project Objectives

The objectives below are stated as they are evidenced by what was actually built and measured, not as aspirations:

- Deliver cuffless blood-pressure estimation accurate enough to be clinically useful, measured against the AAMI and BHS standards — currently achieved for diastolic pressure, not yet for systolic.

- Make blood-pressure measurement a guided, low-friction routine through scheduled reminders and a step-by-step in-app flow, rather than a manual, easy-to-skip task.
- Give caregivers near-real-time visibility into a linked patient’s vitals, medication adherence, and risk alerts without the patient having to actively share anything after the initial link.
- Improve real-world blood-pressure accuracy per patient via a calibration mechanism that learns from occasional reference cuff readings.
- Reduce missed medication doses via scheduled reminders plus an optional physical LED/buzzer indicator on a smart medicine box.
- Provide a symptom-triage capability that can stand alone or be triggered directly from a concerning blood-pressure reading, with a safety-first rule engine that can escalate urgency independently of the ML prediction and never let the model suppress a red flag.
- Support Arabic and English as first-class, parallel experiences rather than a translated afterthought.

1.2 Contribution to Scientific Discovery

The project’s technical contribution centers on two applied machine-learning efforts, and both are documented here with their real, measured results rather than a single flattering headline number chosen after the fact:

- A **cuffless blood-pressure estimation pipeline** that trains and fairly compares ten different models — six classical regressors and four deep-learning architectures — on an identical, record-level split of a MIMIC-derived dataset of over 650,000 signal windows. The headline result runs directly against the intuitive expectation going into the project: the team’s own purpose-built architecture, which combines a temporal convolutional network encoder, a bidirectional LSTM, and multi-head self-attention specifically to capture rich temporal structure, did *not* outperform a considerably simpler plain one-dimensional convolutional network trained on the same data with the same features. That result is reported as measured, not explained away, and Chapter 7 discusses why it plausibly happened.
- A **structured symptom-triage model** that deliberately replaces an earlier free-text/TF-IDF representation with a severity-, duration-, age-, and vitals-aware structured feature vector, precisely because a bag-of-words representation has no way to encode “a mild headache for two days” as different from “a severe headache for two weeks.” Evaluating this new representation against simpler baselines trained on the identical data surfaces a second counter-intuitive finding: a plain Logistic Regression baseline scores higher than the LightGBM model that was actually chosen for production. Encountering the same pattern independently in two unrelated subsystems is itself a transferable finding, not a coincidence to be explained away in either case.

- A **per-patient calibration subsystem** for the blood-pressure model that adapts a population-trained model to one specific individual without ever retraining the model itself — starting from a noise-resistant median offset and automatically upgrading to a full linear fit once enough reference readings accumulate, with a separate scheduled job that watches for the fit drifting out of date against the patient’s own history. This is evaluated here against its own honestly inconclusive simulation result (Section 7.3) rather than assumed to work simply because the mechanism is well-designed.
- **Methodological discipline that eliminates a whole class of bug by construction.** For both AI subsystems, the exact same preprocessing/feature-engineering code that produced the training data is imported directly by the production inference path, rather than reimplemented a second time. This guarantees that what the model sees in production is mathematically identical to what it saw during training — a guarantee that is usually only aspirational in applied ML systems, where training and inference code drift apart silently over time as one side is updated and the other is not.

Chapter 2

Related Work

2.1 Cuffless Blood Pressure Estimation

Cuffless blood-pressure estimation from photoplethysmography (PPG) and electrocardiography (ECG) signals is a well-known hard problem in biomedical signal processing, and the physiological basis for attempting it at all is real: the *pulse transit time* (PTT) — the delay between the electrical activation of the heart, seen as the ECG R-peak, and the arrival of the resulting pressure wave at a peripheral site, seen as the PPG peak — is inversely related to arterial pressure, because a stiffer, more pressurized artery propagates a pulse wave faster than a more compliant one. Early cuffless approaches, such as Kachuee et al. [2], built explicit regression models directly on PTT and a small set of other hand-crafted waveform features. The persistent difficulty with this line of work is that the relationship between these features and true blood pressure is nonlinear, patient-dependent, and confounded by vascular tone, age, and the specific measurement site, which caps the accuracy any purely feature-based model can reach across a diverse population — a ceiling this project’s own results reproduce (Section 6.1.3 and Chapter 7).

More recent work has shifted toward deep learning applied directly to the raw waveform, letting the model learn morphological features beyond hand-crafted PTT rather than depending entirely on it [3, 4]. A recurring caveat across essentially all of this literature is dataset provenance: the most common public benchmark for this problem is derived from the MIMIC critical-care database [1], whose arterial-line reference pressures come from ICU patients on invasive monitoring, not from ambulatory or otherwise healthy subjects. A model trained on that distribution does not automatically generalize to the resting, largely healthy or moderately hypertensive home-use population that a wearable consumer product actually targets — a limitation this project inherits directly, and states plainly rather than glossing over (Chapter 7).

This project’s dataset handling and general preprocessing pipeline — bandpass filtering, resampling to a common rate, and peak-based reference-label extraction from the arterial-line waveform itself — follow the same broad approach already established across the MIMIC-based cuffless-BP literature, rather than introducing a novel signal-processing method of its own. The project’s genuine contribution sits one level up from preprocessing: a controlled, ten-model comparison run on an identical record-level split so that every model is judged on exactly the same evidence; a fault-tolerant, resumable training pipeline built to survive a shared, quota-limited GPU environment; and a per-patient calibration layer built on top of the population-trained model, addressing the generalization gap above not by collecting more diverse training data (which was not available) but by adapting the deployed model to each individual patient after the fact.

2.2 Rule-Based Clinical Decision Support

On the symptom-checker side, rule-based clinical decision support — red-flag detection and escalation-only urgency rules layered on top of a statistical classifier — is an established, deliberately conservative design pattern in clinical software. Its value is specific and well understood: it lets a statistical model’s errors be caught by a simpler, fully auditable safety net, one that a clinician can read, reason about, and sign off on line by line in a way that a gradient-boosted decision ensemble’s internal weights cannot be. The project’s own rule engine (Section 8.4.1) follows this established principle rather than inventing a new safety paradigm, and is deliberately implemented as pure, independently testable functions kept entirely separate from the ML classifier’s code path — so that the safety layer’s behavior is fully determined by its documented inputs and never by anything the model happens to have learned.

The specific modeling choices also sit within the current mainstream of applied machine learning: a gradient-boosted tree model (LightGBM [5]) for the structured symptom classifier, and a direct comparison against a modern attention-based architecture [6] on the blood-pressure side. What is less common is reporting, without spin, that the simpler alternative won in both comparisons. That finding is a genuinely useful, honestly-measured data point for the broader and still-open question of when additional architectural sophistication is actually justified by the data available, rather than assumed to help by default.

2.3 Internal Comparative History

The project’s own internal history is itself a form of comparative evidence, and is worth reading as a timeline rather than a single decision. Two earlier, larger, less-regularized blood-pressure model attempts — referred to internally in the training notebook as “v4” and “v10” — are documented directly alongside the final results: v4 used no L2 regularization at all and produced an 86% train/validation MAE gap, a textbook case of severe overfitting; v10 attempted to fix class imbalance by discarding real training windows down to an equal count per class, which threw away most of the usable dataset and made the model measurably worse overall rather than better. Both failures directly and traceably shaped the final configuration used in this project: inverse-frequency sample weighting in place of downsampling, so no real data is thrown away, and a lighter, carefully re-tuned regularization scheme that closes most of the overfitting gap without under-fitting the model. Retaining this lineage of failed and refined attempts in the documentation, rather than presenting only the final configuration as if it were obvious from the start, is a more honest and more useful account of the actual engineering process than a single final number ever could be.

Chapter 3

Software Description

3.1 Software Architecture

Health Mate is composed of four cooperating parts: a Flutter mobile client serving both patient and caregiver roles, a FastAPI backend that owns all business logic and state, two purpose-built AI models, and two independent IoT devices. A managed WebRTC voice/video calling layer connects patients and caregivers directly. Figure 3.1 shows the full architecture and the protocols each edge uses.

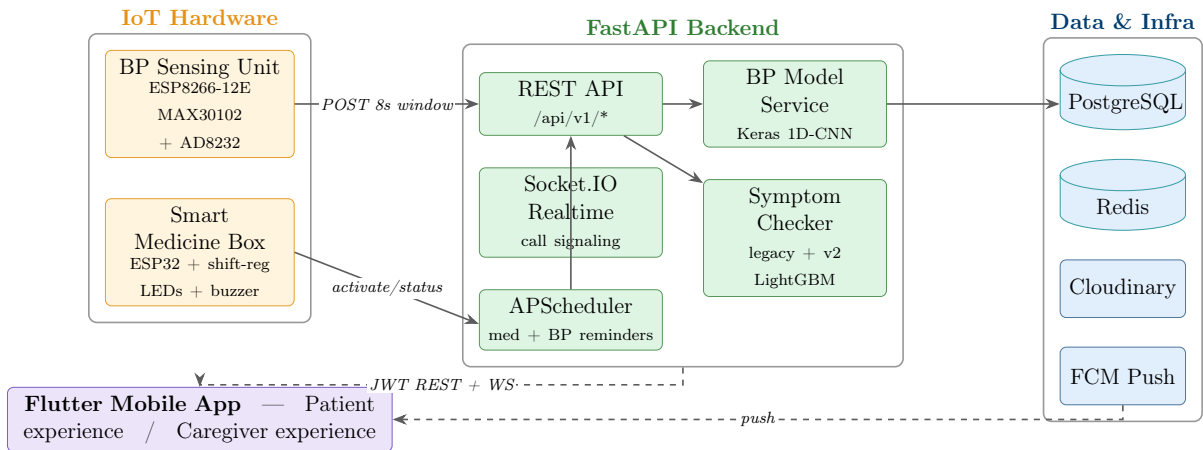


Figure 3.1: Health Mate system architecture. The hardware never makes clinical decisions — every safety-relevant step runs in the backend.

The system is deliberately split so that **the hardware never makes clinical decisions**. Both the blood-pressure unit and the smart box are data collectors / actuators only: the ESP8266 firmware buffers 800 raw PPG/ECG samples, checks finger/lead gating, and posts the window to the backend; it contains no model, no calibration, and no risk logic. Every clinically meaningful step — signal validation, model inference, calibration, drift detection, risk classification, and alerting — runs in the backend. This is a verifiable property of the code, not merely stated intent.

3.1.1 Two Architectural Styles, Side by Side

The backend mixes two architectural styles, and it is worth being explicit about that rather than pretending it is uniform:

- **Service-layer style** (most of the system). Authentication, medications, vitals, noti-

fications, contacts, calls, IoT device management, and the BP reminder/calibration/-drift logic all follow a straightforward pattern: FastAPI routers call flat service modules, which talk directly to SQLAlchemy models. There is no repository abstraction here — this is simple, easy to trace, and dominant across the codebase.

- **Clean/hexagonal architecture** (symptom checker v2). The newer symptom-checker implementation is structured as a deliberate, documented refactor with a genuine dependency-inversion boundary (Figure 3.2). Use cases depend on domain interfaces, and a single dependency-injection module decides which concrete infrastructure implementation satisfies each interface.

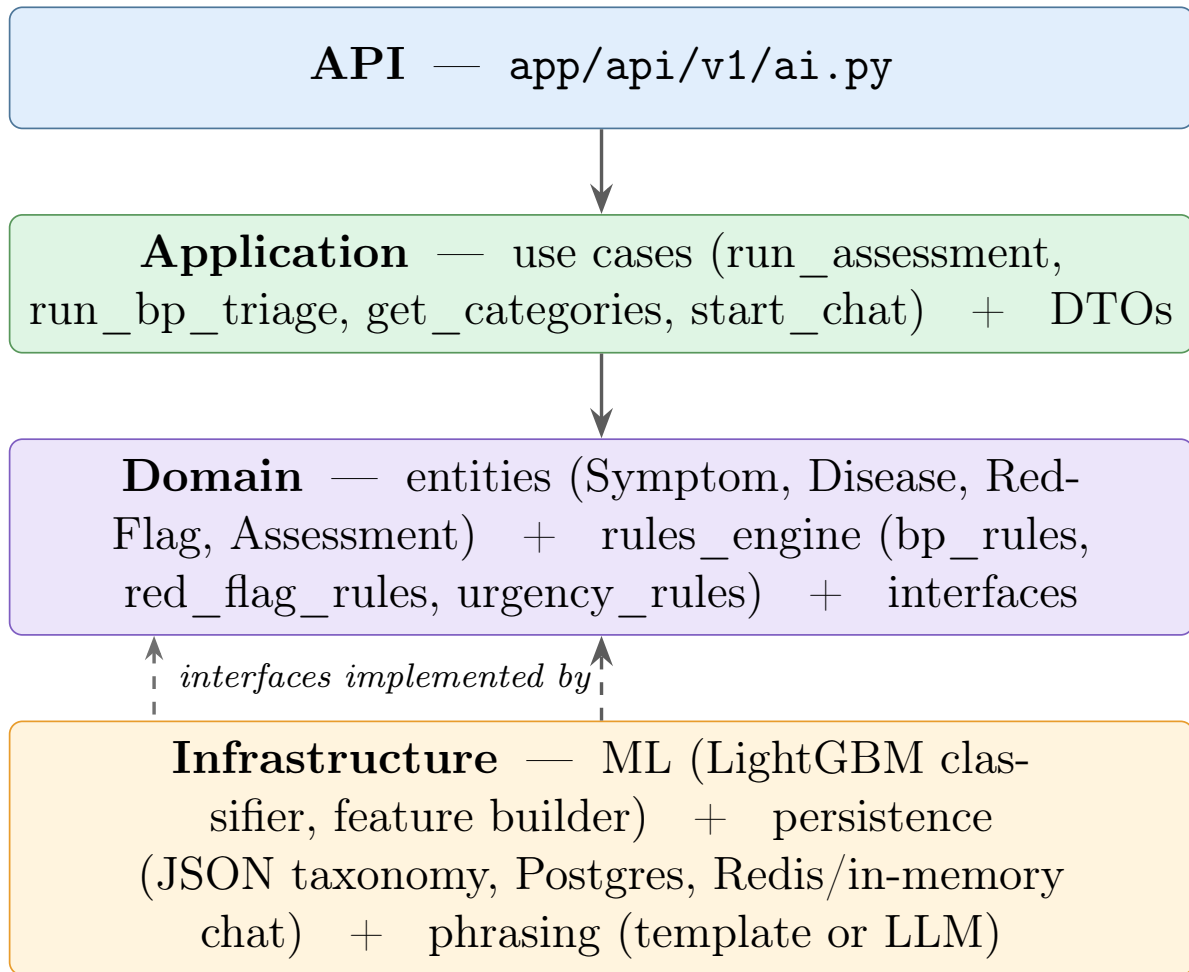


Figure 3.2: Clean-architecture layering used for the symptom checker v2: the only subsystem restructured this way. Dependencies point inward; infrastructure implements domain interfaces, wired in one dependency-injection module.

This layered structure exists *only* for the symptom checker. An older, simpler implementation is still loaded at application startup alongside the v2 path; the two coexist, gated by a backend setting. The point here is architectural: this is the one part of the backend deliberately restructured, and the rest has intentionally been left in its simpler, already-working form.

3.1.2 Cross-Cutting Backend Pieces

- **Configuration** — a single Pydantic **Settings** object sourced from environment variables, covering the database, Redis, JWT, IoT mode, AI model paths, Cloudinary, Firebase, WebRTC STUN/TURN servers, and CORS. No hardcoded configuration values live in application code.
- **Security** — JWT issuance and verification (access + refresh tokens), and bcrypt password hashing.
- **Scheduler** — APScheduler background jobs. Medication reminders are cron-style jobs; when one fires, the backend looks up matching medications, optionally activates smart-box drawers, and sends a notification. BP reminders use the same mechanism.
- **Realtime layer** — a `python-socketio` async server mounted onto the FastAPI app, authenticated with the same JWT, used specifically for WebRTC call signaling.
- **Firebase Cloud Messaging** — push notifications (medication alarms, BP reminders, critical-vitals alerts), separate from the Socket.IO channel.
- **PostgreSQL + Alembic** — the system of record, with a migration history that traces the project's real build-out.

3.2 Tools and Technologies Used

The technology choices below were made for concrete, stated reasons rather than by default (Table 3.1).

Table 3.1: Technology stack and rationale.

Layer	Technology	Why chosen
Mobile client	Flutter, Riverpod, Dio + Retrofit, Hive, flutter_secure_storage, easy_localization	Cross-platform single codebase; Riverpod family providers let one set of screens serve both patient and caregiver views; Hive gives offline-first access to recent vitals
Backend	FastAPI (async), SQLAlchemy (async, asyncpg), Alembic, APScheduler, python-socketio	Non-blocking I/O suited to real-time telemetry; APScheduler drives reminders; Socket.IO handles WebRTC call signaling
Data	PostgreSQL 15, Redis	Postgres as the relational system of record; Redis for app-level caching and chat-session state
BP model	TensorFlow/Keras, scikit-learn, XGBoost, LightGBM, CatBoost	A deliberately broad model comparison rather than committing to one architecture upfront
Symptom checker	LightGBM (v2, structured), scikit-learn (v1, TF-IDF, legacy)	LightGBM for the new structured feature set; the older pipeline retained, not deleted
Notifications	Firebase Cloud Messaging, flutter_local_notifications	Cloud push for reliability, local scheduling for offline reminder delivery
Media	Cloudinary	Profile and medication/adherence photo uploads
Calling	flutter_webrtc, Socket.IO signaling	Direct patient/caregiver audio/video without a third-party calling SDK
Deployment	Docker Compose	Single-host development/demo deployment
Firmware	Arduino / C++ (ESP8266, ESP32)	Low-cost, WiFi-capable microcontrollers matched to the model's 100 Hz input contract

3.3 Software Functionalities

The features below are the ones that actually exist in the codebase; the use-case catalogue in Table 3.4 records their implementation evidence, and the use-case diagram in Figure 3.3 groups them by actor.

- **Authentication** — email/password and Firebase (social) sign-up and login, JWT access + refresh tokens, role selection at registration.
- **Patient–caregiver linking** — link by user ID (with a QR-code UI on top), multiple caregivers per patient, one designated primary caregiver, and unlinking.
- **Guided blood-pressure measurement** — an 8-second sensor capture walked through step by step, feeding the backend's model + calibration pipeline, with results withheld

from the patient as a raw number until calibration is trustworthy.

- **Per-patient calibration and drift detection** — a two-tier fit (robust median offset below 8 samples, full linear regression above), with a scheduled job flagging sustained drift or staleness.
- **BP measurement reminders** — up to three daily reminder times derived from one patient-chosen start time.
- **Medication management** — catalog with dosage/schedule, optional smart-box drawer assignment, scheduled alarms (push + local notification + optional LED/buzzer), adherence logging with optional photo proof.
- **Structured symptom triage** — a severity/duration/age/ vitals-aware assessment wizard backed by a rule engine, plus a direct handoff from a concerning BP reading into the same triage flow.
- **Emergency alerting** — automatic caregiver push on high/critical BP or red-flag symptoms, rate-limited per risk tier.
- **Medical contacts** — doctor/clinic/pharmacy/emergency/family contact list per patient, with the caregiver auto-added as a family contact on linking.
- **Audio/video calling** — WebRTC calls between a patient and a linked caregiver, with dedicated incoming-call and emergency-call screens.
- **Notifications inbox** — in-app list of all past notifications, read/unread state.
- **Localization** — full English/Arabic parity with RTL support.
- **Offline support** — cached last-known BP reading and history (Hive-backed); reminders continue to fire locally without connectivity.

3.4 Functional Requirements

The functional requirements in Table 3.2 summarize what the implemented system must do from a user and system-behavior perspective.

Table 3.2: Functional requirements.

Requirement	Implemented behavior
Account registration and login	Patients and caregivers can create accounts, sign in, refresh sessions, and use Firebase/social authentication.
Role-based experience	The mobile application opens patient or caregiver workflows according to the saved account role.
Patient-caregiver linking	Patients and caregivers can link, unlink, and maintain one active primary caregiver for a patient.
Guided BP measurement	The BP device sends an 800-sample PPG/ECG window; the backend stores a result or a clear rejection reason.
Calibration and cuff confirmation	The backend asks for cuff confirmation when calibration is missing, weak, or due, then stores calibration samples.
Vitals history and monitoring	Patients and linked caregivers can view recent readings, history, and statistics.
BP reminders	Patients can schedule daily blood-pressure reminders from a selected start time.
Medication management	Patients can add, edit, delete, schedule, and assign medications to smart-box drawers.
Medication adherence	Patients can confirm medication intake, with optional image proof.
Smart-box activation	The backend or mobile app can activate drawer LEDs and buzzer when the smart box is reachable.
Symptom assessment	The app collects structured symptoms, severity, duration, age group, known conditions, and vitals, then returns urgency and action guidance.
Emergency notifications	Caregivers receive alerts for critical BP or high-risk symptom assessments, with cooldown protection.
Medical contacts	Patients can manage doctor, clinic, pharmacy, emergency, and family contacts.
Audio/video calling	Linked patients and caregivers can start, accept, reject, and end WebRTC calls.
Notification inbox	Users can list, read, mark, and delete notifications.
Localization	The mobile app supports English and Arabic, including RTL display for Arabic.

3.5 Non-Functional Requirements

The non-functional requirements in Table 3.3 describe the quality constraints that shape the implementation.

Table 3.3: Non-functional requirements.

Quality attribute	Implementation approach
Safety	Hardware devices collect signals or actuate alarms only; clinical decisions run in the backend.
Security	Human endpoints use JWT authentication, passwords are hashed, and BP hardware endpoints use device credentials.
Privacy	Health records, contacts, calls, notifications, and calibration data are scoped to the owning patient and authorized caregivers.
Reliability	Model-loading failures, poor signals, unreachable devices, and notification failures degrade gracefully.
Data integrity	PostgreSQL foreign keys, enums, constraints, and the primary-caregiver unique index protect the data model.
Maintainability	The project separates Flutter UI/data layers, FastAPI routers/services/models, migrations, and symptom-checker use cases.
Scalability	Async FastAPI/SQLAlchemy, PostgreSQL, and Redis support non-blocking API, persistence, and cache/session state.
Usability	Guided mobile flows reduce user mistakes during BP measurement, medication setup, symptom triage, and caregiver monitoring.
Localization	English and Arabic interfaces are supported, including RTL presentation for Arabic.
Portability	Flutter supports multi-platform mobile development, and Docker Compose provides repeatable backend deployment.
Auditability	Security and compliance events are stored in audit logs where available.
Offline tolerance	Recent vitals/history are cached and local reminder behavior can continue without connectivity.

Table 3.4: Professional use-case catalogue for the implemented Health Mate system.

Primary actor	Use case	Implementation evidence and outcome
Patient	Register / sign in	FastAPI auth endpoints issue JWT access/refresh tokens; Flutter stores credentials securely and gates patient/caregiver navigation.
Patient, caregiver	Link / unlink accounts	<code>patient_caregiver_links</code> stores active patient-caregiver relationships, with one active primary caregiver per patient.
Patient	Take guided BP measurement	The BP device submits an 800-sample PPG/ECG window; the backend runs inference, calibration, status handling, and persistence.
Patient	Confirm cuff reading	Cuff confirmation completes pending readings and appends calibration samples for future per-patient correction.
Patient, caregiver	View vitals history	Vitals history and stats endpoints power patient charts and caregiver monitoring screens for linked patients.
Patient	Manage medications	Medication CRUD stores dosage, schedule, optional image, and optional smart-box drawer assignment.
Patient	Log adherence	Medication adherence records a taken timestamp and optional image proof rather than trusting the box as a physical retrieval sensor.
Patient	Run symptom assessment	The v2 structured assessment path combines selected symptoms, severity, duration, age group, vitals, red flags, and ML predictions.
Caregiver	Receive alerts	Notification services send caregiver alerts for critical BP and red-flag/high-urgency symptom assessments, with cooldown logic for BP tiers.
Patient, caregiver	Audio/video call	Call sessions track caller/callee, call type, status, offer/answer SDP, and lifecycle events used by the WebRTC UI.
Patient	Manage medical contacts	Contacts endpoints persist doctors, clinics, pharmacies, emergency contacts, and family contacts under the patient account.

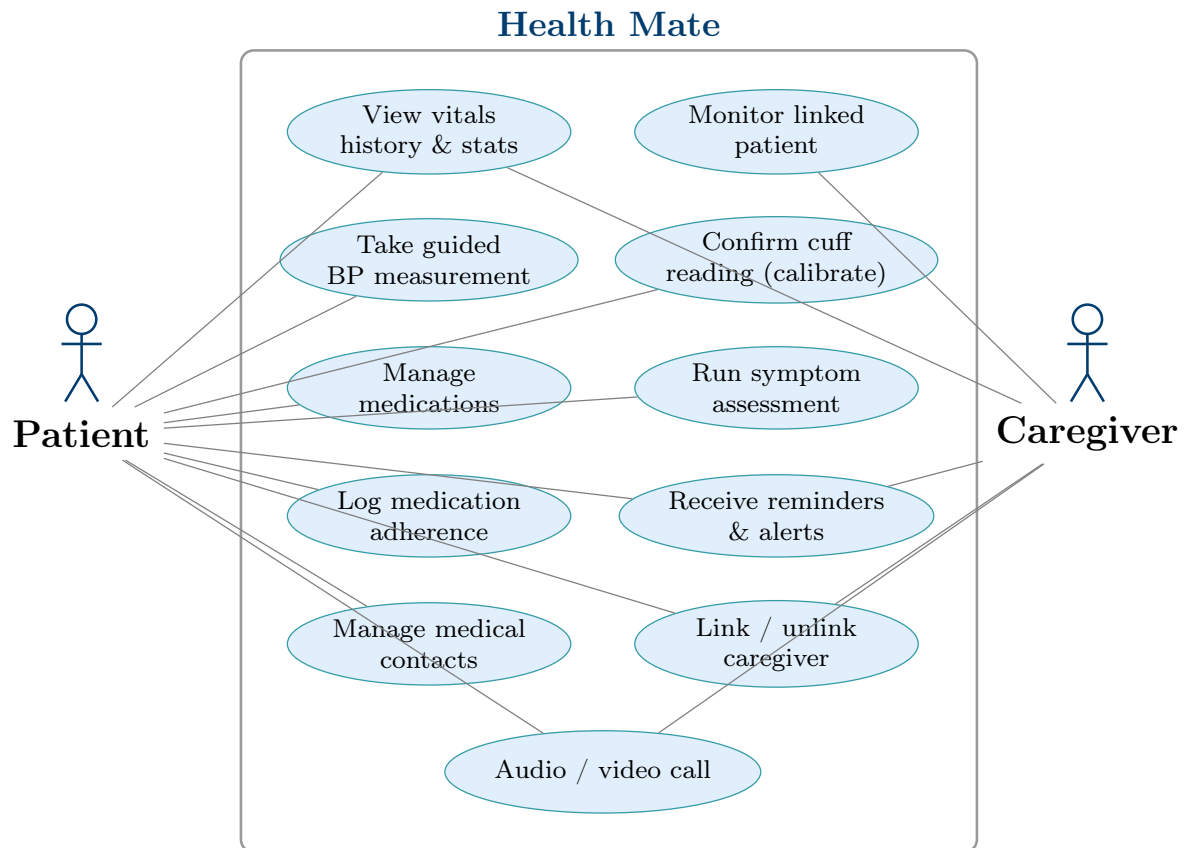


Figure 3.3: Use-case diagram. Patients and caregivers share the calling, linking, alerting, and monitoring use cases; measurement, calibration, medication, and symptom assessment are patient-driven, while “monitor linked patient” is caregiver-specific.

Chapter 4

IoT Hardware Subsystems

Health Mate has two entirely separate hardware subsystems — a blood-pressure sensing unit and a smart medicine box — built on different microcontrollers, different firmware, and different interaction models. They deliberately do not share a microcontroller. Table 4.1 summarizes the contrast.

Table 4.1: The two IoT subsystems at a glance.

	BP sensing unit	Smart medicine box
MCU	ESP8266-12E NodeMCU	ESP32
Sensors / actuators	MAX30102 (PPG, I2C), AD8232 (ECG, analog + lead-off pins)	74HC595 shift register driving one LED per drawer, plus a buzzer — no motors
Talks to backend how	Pushes data (POST /bp/submit, /bp/heartbeat)	Receives commands (POST /activate)
Authentication	Device ID + hashed token (real mechanism)	None — plain HTTP, same-network trust
Actuation	None (pure sensor)	LEDs + buzzer only — no physical dispensing
Failure mode	Retry with backoff; abort on auth rejection	Fail-soft: errors logged, alarm job continues

4.1 Blood-Pressure Sensing Unit

4.1.1 Hardware

- **ESP8266-12E NodeMCU** — WiFi microcontroller, no on-device model inference.
- **MAX30102** — PPG sensor over I2C, configured with `sampleAverage = 1`, chosen specifically to produce raw, unaveraged 100 Hz output matching the trained model’s expected input rate.
- **AD8232** — single-lead ECG front-end, analog output on A0, with dedicated lead-off detection pins and a shutdown pin held high to keep the chip enabled.

The full breadboard wiring, the complete pin-connection tables for both sensors, and the electrode-placement reference are shown in Figure 4.1. The MAX30102 communicates

over I2C (SDA on D2, SCL on D1); the AD8232’s analog output is read on A0, its shutdown pin is driven from D5, and its two lead-off detection lines are on D6/D7. A common ground between all three modules is required. Figure 4.2 shows the standard three-lead electrode placement on the body in more anatomical detail.

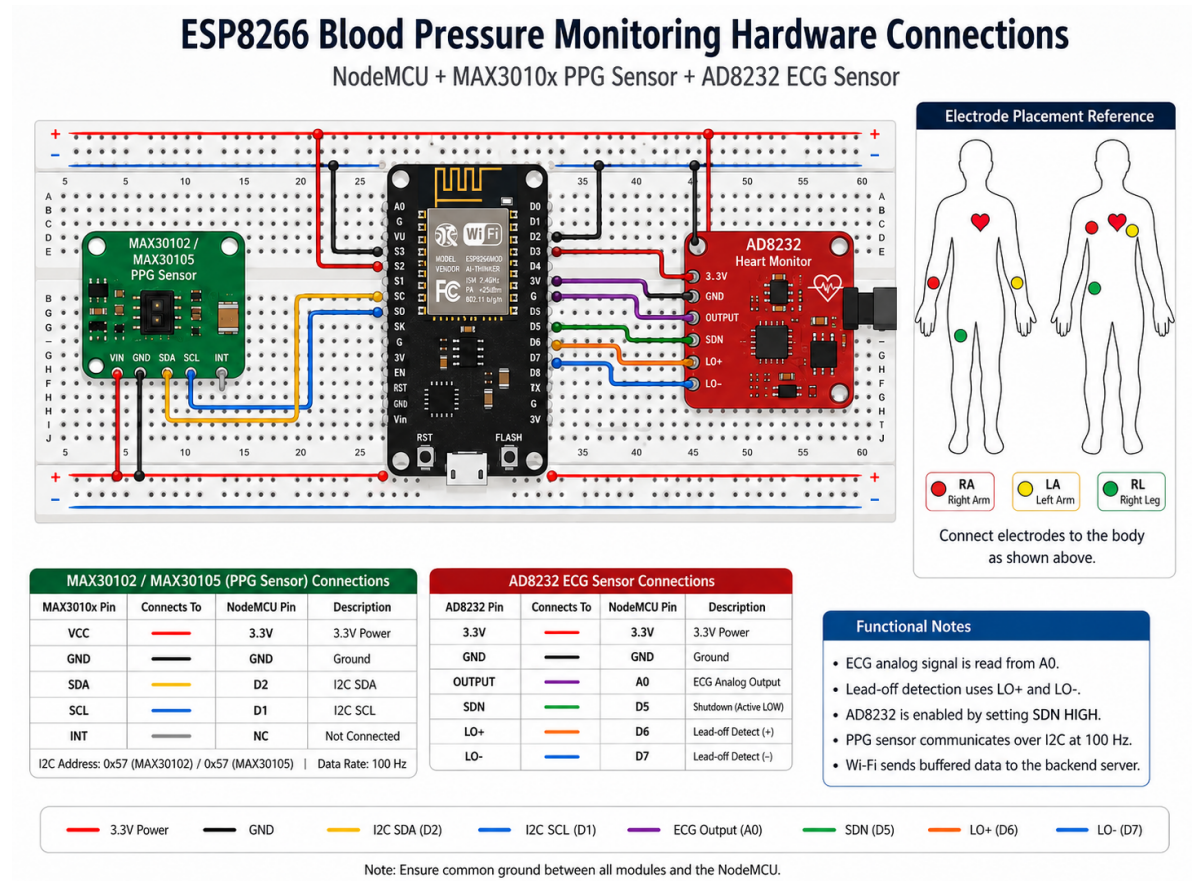


Figure 4.1: Blood-pressure sensing unit — complete hardware wiring (NodeMCU ESP8266 + MAX30102 PPG + AD8232 ECG), with per-sensor pin tables, electrode placement, and functional notes.

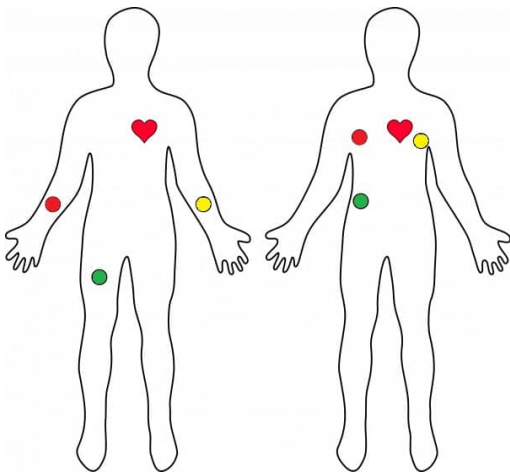


Figure 4.2: AD8232 three-lead (RA/LA/RL) electrode placement on the torso and limbs, shown in two equivalent placement variants.

4.1.2 Firmware Behavior

This is real, working firmware, not a wiring test. The behavior below is confirmed directly in the source:

- Samples both channels at a precise 100 Hz, buffering exactly 800 samples per channel — an 8-second window matching the model’s input contract.
- **Gates on signal presence** before accepting any sample: both ECG leads connected *and* a finger detected on the PPG sensor ($IR > 50,000$ counts). If gating fails for 50 consecutive checks (~ 500 ms), the in-progress buffer is discarded rather than silently accepting a bad window.
- Computes **on-device SpO2** (ratio-of-ratios) and **heart rate** (peak detection), included in the payload only when physiologically plausible — an unusable value is omitted, not sent as a placeholder.
- **Uploads the completed window** via HTTP POST with device-ID and device-token headers and an exponential-backoff retry policy (up to 4 attempts), except on a 401/403/422 response, treated as a permanent rejection and aborted immediately.
- **Sends a heartbeat** on a 10-second timer, or immediately (at most once per second) whenever the finger/lead state changes, so the backend’s device flags reflect a mid-measurement disconnection within about a second.

A real, unresolved gap: the device token in the current firmware is a hardcoded placeholder string, not a per-device provisioned secret. The authentication *mechanism* (hashed token compared server-side) is real and correct, but the credential shipped in this firmware file is not production-grade.

4.2 Smart Medicine Box

4.2.1 Hardware — an Explicit Correction

The smart box is an ESP32 running its own tiny web server on port 80, a 74HC595 shift register driving up to 6 LEDs (one per drawer), and a buzzer. **There is no motor, servo, solenoid, or physical locking/dispensing mechanism anywhere in the firmware.** Concretely, the smart box *indicates* which drawer to open — it does not open it. On receiving an activation payload such as `{"drawers": [3]}`, it lights the corresponding LED(s) and runs an intermittent buzzer alert; an empty list turns everything off. The patient still physically opens the correct drawer once the LED points to it. Any description of this as “automatically dispensing” overstates the hardware; it is a guided-indicator system — still a real and useful piece of assistive hardware for an elderly or memory-impaired patient, but a materially simpler and more reliable mechanism than a motorized dispenser.

The physical build and full wiring are shown in Figure 4.3: a laser-cut wooden enclosure with six medication compartments and an acrylic window, an ESP32 driving a

74HC595 shift register (data on GPIO23, clock on GPIO18, latch on GPIO5), the register's Q0–Q5 outputs lighting the six drawer LEDs through current-limiting resistors, and an active buzzer on GPIO4.

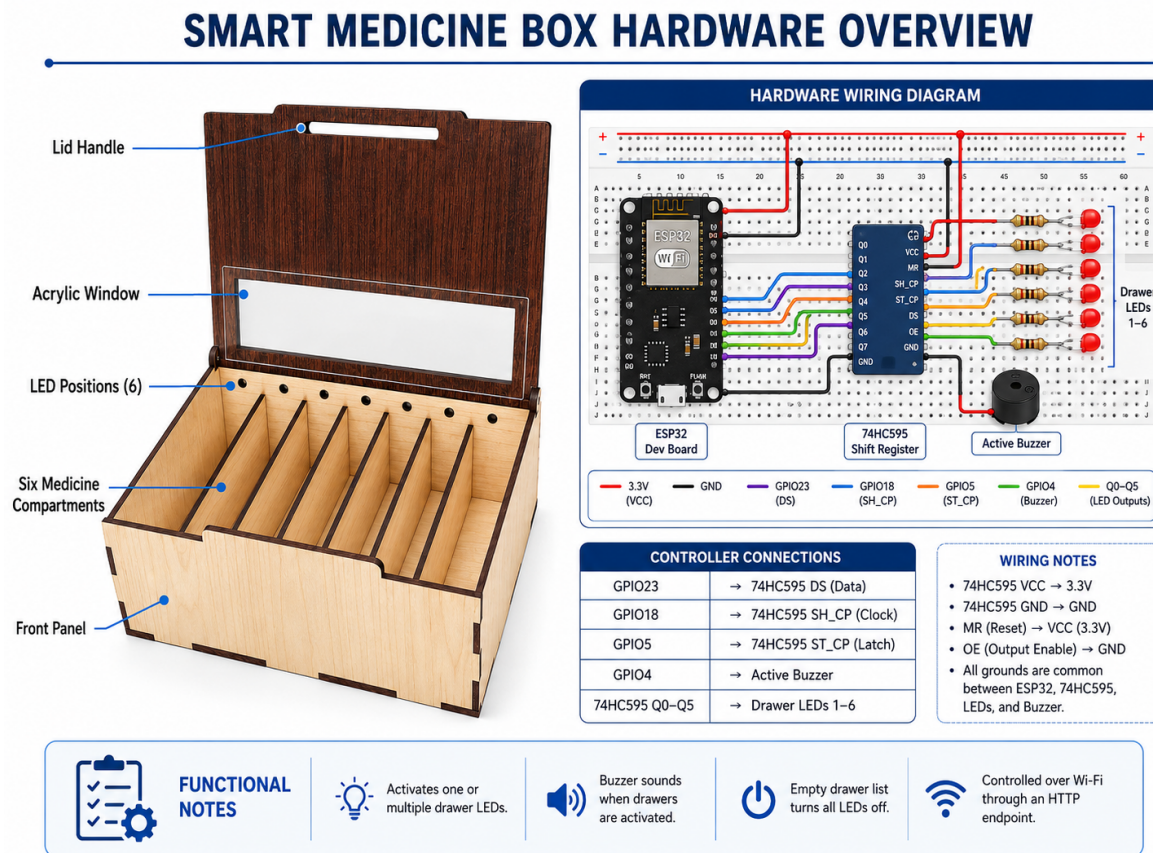


Figure 4.3: Smart medicine box — physical enclosure (six compartments, LED positions, acrylic window) and complete hardware wiring (ESP32 + 74HC595 shift register + six drawer LEDs + active buzzer), with the controller pin map and functional notes.

4.2.2 Who Talks to It

Two independent callers can activate the box:

1. **The backend scheduler** — when a medication alarm fires, it collects the drawers due at that time and calls `activate()`; the call is fail-soft, so an unreachable box never crashes the alarm job.
2. **The Flutter app directly** — it can make its own HTTP call straight to the ESP32's local IP, bypassing the backend, as long as the phone is on the same network as the box.

4.2.3 Reliability Characteristics

- **No acknowledgment of physical retrieval** — because no sensor detects whether a drawer was opened, adherence is logged from the patient explicitly confirming “taken”

in the app, not from any hardware signal.

- **Offline behavior** — if the ESP32 is unreachable when an alarm fires, the LED/buzzer step is silently skipped, but the phone push/local notification for the same reminder is unaffected.
- **Multiple medications at the same time** are collected into one activation call, lighting multiple LEDs simultaneously.

Chapter 5

Database Design

Health Mate’s system of record is PostgreSQL 15, accessed through SQLAlchemy’s async ORM and versioned with Alembic. The schema comprises 21 model classes. Figure 5.1 shows the core entities and their relationships; Table 5.1 lists every table.

5.1 Entity-Relationship Diagram

The ERD in Figure 5.1 is the project-maintained diagram exported from the implemented backend schema. Because the source is a wide diagram, it is placed as one complete landscape page instead of being split into cropped pieces. In print, this page should be printed in landscape orientation.

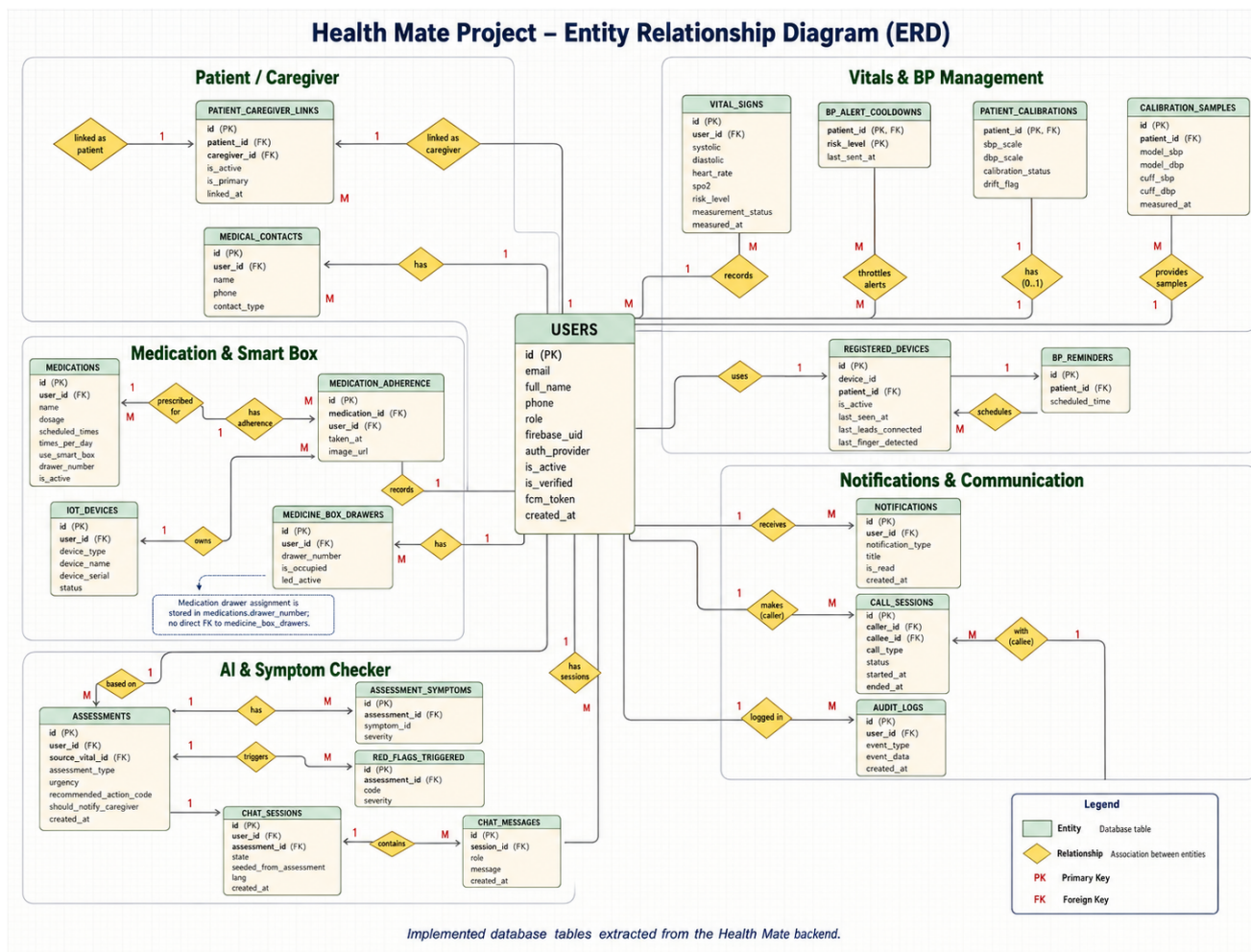


Figure 5.1: Complete entity-relationship diagram for the implemented Health Mate backend schema.

5.2 Tables at a Glance

Table 5.1: The 21 tables of the Health Mate schema.

Table	Purpose	Notable constraint
users	Patients and caregivers (one table, role discriminates)	unique email, firebase_uid
patient_caregiver_link	Many-to-many patient↔caregiver linking	partial unique index: one active primary per patient
vital_signs	BP/HR/SpO2 readings, sensor or manual	FK user_id (cascade)
bp_alert_cooldowns	Prevents duplicate emergency alerts per tier	composite (patient_id, risk_level)
medications	Medication catalog + schedule + drawer assignment	FK user_id
medication_adherence	“Taken” confirmations, optional photo proof	FK medication_id, user_id
medical_contacts	Doctor/clinic/pharmacy/emergency/family contacts	FK user_id
notifications	All push/in-app notification records	FK user_id
call_sessions	WebRTC audio/video call state	FK caller_id, callee_id
iot_devices	Generic sensor/medicine-box registry	unique device_serial
medicine_box_drawer	Per-drawer occupancy/LED state	FK user_id
registered_devices	BP-hardware auth (device_id + hashed token)	unique device_id
patient_calibrations	Per-patient BP calibration coefficients	PK = patient_id (1:1)
calibration_samples	Historical cuff-vs-model reading pairs	FK patient_id
bp_reminders	Scheduled daily BP reminder times	FK patient_id
audit_logs	Security/compliance event trail	FK user_id (nullable, SET NULL)
assessments	Symptom-checker v2 assessment results	FK user_id?, source_vital_id?
assessment_symptoms	Symptoms selected within one assessment	FK assessment_id (cascade)
red_flags_triggered	Red flags fired within one assessment	FK assessment_id (cascade)
chat_sessions	Symptom-checker chat session state (model exists)	FK user_id?, assessment_id?

Table 5.1 continued

Table	Purpose	Notable constraint
chat_messages	Individual chat turns	FK session_id (cascade)

The full visual table reference is placed as one complete page in Figure 5.2. The same information is also summarized in Table 5.1, so the visual figure can remain an overview artifact without replacing the readable text table.

Health Mate Project - Complete Database Tables

Implemented PostgreSQL tables extracted from the backend models

PK Primary Key FK Foreign Key UK Unique Key

Users / Auth

USERS

Stores both patients and caregivers.

Column	Type	Key / Notes
id	UUID	PK, indexed, default uuid
email	String(255)	required, unique, indexed
hashed_password	String(255)	required
firebase_id	String(255)	nullable, unique, indexed
auth_provider	Enum(String)	email google
email_verified_at	DateTime	nullable
last_name	String(255)	required
phone	String(50)	nullable
birth_date	String(50)	nullable
gender	String(50)	nullable
profile_image_url	String(500)	nullable
role	Enum	patient caregiver
is_active	Boolean	default true
is_verified	Boolean	default false
role_name	String(500)	nullable
created_at	DateTime	default now
updated_at	DateTime	auto update
last_login_at	DateTime	nullable

Vitals / BP

PATIENT CALIBRATIONS

Stores current BP calibration coefficients and drift data per patient.

Column	Type	Key / Notes
patient_id	UUID	FK -> users.id
avg_systolic	Float	default 1.0
avg_diastolic	Float	default 0.8
avg_pulse	Float	default 1.0
avg_offset	Float	default 0.0
samples_count	Integer	default 0
last_calibrated_at	DateTime	nullable
calibration_status	String(50)	default not_calibrated
baseline_low_mean_bp	Float	nullable
baseline_low_max_bp	Float	nullable
drift_flag	Boolean	default false
drift_reason	String(50)	nullable
drift_flagged_at	DateTime	nullable
last_drift_calibration_at	DateTime	nullable

Medication / Smart Box

MEDICATIONS

Stores patient medication, dose, schedule and smart-box drawer assignment.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	FK -> users.id, indexed
name	String(255)	required
dosage	String(255)	required
instructions	Text	nullable
image_url	String(500)	nullable
times_per_day	Integer	required
scheduled_times	Array(String)	HH:MM values
is_smart_box	Boolean	default false
drawer_number	Integer	nullable, expected 1-6
is_active	Boolean	default true
created_at	DateTime	default now
updated_at	DateTime	auto update

Notifications / Communication

NOTIFICATIONS

Stores image and push notification records.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	FK -> users.id, indexed
notification_type	Enum	required
title	String(255)	required
message	Text	required
data	JSON	nullable
is_read	Boolean	default false
created_at	DateTime	indexed, default now
read_at	DateTime	nullable

AI / Symptom Checker

ASSESSMENTS

Stores structured symptom-checker and BP-triage assessment results.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	FK -> users.id, indexed
source_vital_id	UUID	nullable FK -> vitals.id, indexed
assessment_type	String(100)	default symptom_checker
category_id	String(100)	nullable, indexed
keywords_tags	String(50)	nullable
pop_group	String(50)	nullable
known_conditions	JSONB	default []
vitals	JSONB	nullable
top_predictions	JSONB	default []
urgency	String(20)	low moderate high critical
urgency_label	String(100)	required
recommended_action_code	String(100)	required
patient_message	Text	required
caregiver_summary	Text	required
should_notify_caregiver	Boolean	default false
should_reassess	Boolean	default false
disclaimer	Text	required
lang	String(5)	default en
created_at	DateTime	indexed, default now

Patient / Caregiver

PATIENT-CAREGIVER LINKS

Associates tables linking patients and caregivers.

Column	Type	Key / Notes
id	UUID	PK, default uuid
patient_id	UUID	FK -> users.id, indexed
caregiver_id	UUID	FK -> users.id, indexed
is_active	Boolean	default true
is_primary	Boolean	default false
linked_at	DateTime	default now
updated_at	DateTime	auto update
is_patient_active_primary_caregiver	Boolean	unique active primary caregiver per patient

Vitals / BP

VITAL SIGNS

Stores BP, HR, SpO2, prediction, sensor data and calibration output.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	FK -> users.id, indexed
systolic	Integer	nullable
diastolic	Integer	nullable
heart_rate	Integer	nullable
spo2	Integer	nullable
risk_level	Enum	normal low moderate high critical
source	String(50)	default sensor
confidence	Float	nullable
signal_quality	Float	nullable
measurement_status	Enum	completed completed_pending rejected
device_id	String(100)	nullable
expected_readings	String(50)	finger-reading leads_off poor_signal
model_systolic	Float	nullable
model_diastolic	Float	nullable
calibrated_systolic	Float	nullable
calibrated_diastolic	Float	nullable
calibration_status	String(50)	indexed, default now
measured_at	DateTime	indexed, default now
created_at	DateTime	default now

Vitals / BP

BP REMINDERS

Stores daily BP measurement reminder times.

Column	Type	Key / Notes
id	UUID	PK, default uuid
patient_id	UUID	FK -> users.id, indexed
scheduled_time	String(5)	HH:MM
created_at	DateTime	default now
service_rule	Text	3 residents: start, 4th, 10th

Medication / Smart Box

MEDICATION ADHERENCE

Logs when a medication was confirmed via smart box.

Column	Type	Key / Notes
id	UUID	PK, default uuid
medication_id	UUID	FK -> medications.id, indexed
user_id	UUID	FK -> users.id, indexed
taken_at	DateTime	default now
image_url	String(100)	nullable

Medication / Smart Box

MEDICINE BOX DRAWERS

Stores drawer occupancy and LED data for a medicine box.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	FK -> users.id, indexed
drawer_number	Integer	required
is_occupied	Boolean	default false
log_active	Boolean	default false
created_at	DateTime	auto update
updated_at	DateTime	auto update
note	Text	medications_drawer_number stores assignment; no direct FK

Users / Auth

AUDIT LOGS

Security and compliance event trail.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	nullable FK -> users.id, indexed
event_type	String(100)	indexed
event_description	Text	required
event_date	JSON	nullable
ip_address	String(50)	nullable
user_agent	String(500)	nullable
created_at	DateTime	indexed, default now

AI / Symptom Checker

RED FLAGS TRIGGERED

Stores red flags triggered during a symptom assessment.

Column	Type	Key / Notes
id	UUID	PK, default uuid
assessment_id	UUID	FK -> assessments.id, indexed
code	String(100)	indexed
severity	Integer	required

AI / Symptom Checker

CHAT MESSAGES

Stores chatbot messages inside a symptom chat session.

Column	Type	Key / Notes
id	UUID	PK, default uuid
session_id	UUID	FK -> chat_sessions.id, indexed
role	String(20)	required
message	Text	required
pretext	JSONB	nullable
created_at	DateTime	indexed, default now

Patient / Caregiver

MEDICAL CONTACTS

Stores doctors, clinics, pharmacies, emergency and family contacts.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	FK -> users.id, indexed
name	String(255)	required
phone	String(50)	required
contact_type	Enum	doctor clinic pharmacy emergency family
notes	Text	nullable
created_at	DateTime	default now
updated_at	DateTime	auto update

Vitals / BP

BP ALERT COOLDOWNS

Prevents repeated BP alerts for the same patient and risk level.

Column	Type	Key / Notes
id	UUID	PK, default uuid
patient_id	UUID	FK -> users.id
risk_level	String(50)	FK
last_alert_at	DateTime	default now / auto update

Vitals / BP

CALIBRATION SAMPLES

Historical cuff-vs-model samples used to update patient calibration.

Column	Type	Key / Notes
id	UUID	PK, default uuid
patient_id	UUID	FK -> users.id, indexed
model_bp	Float	required
model_dbp	Float	required
cuff_bp	Float	required
cuff_dbp	Float	required
signal_quality	Float	default 1.0
measured_at	DateTime	default now

Vitals / BP

REGISTERED DEVICES

Authenticates BP hardware devices and maps them to a patient.

Column	Type	Key / Notes
id	UUID	PK, default uuid
device_id	String(100)	unique, indexed
token_hash	String(200)	required
patient_id	UUID	FK -> users.id, indexed
is_active	Boolean	default true
registered_at	DateTime	default now
last_seen_at	DateTime	nullable
last_leads_connected	Boolean	nullable
last_finger_detected	Boolean	nullable

Medication / Smart Box

IOT DEVICES

Generic registry for sensors and medicine boxes.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	FK -> users.id, indexed
device_type	Enum	drug_sensor ecg_sensor medicine_box
device_name	String(255)	required
device_serial	String(255)	unique
status	Enum	connected disconnected unstable
signal_quality	Float	nullable
last_ping_at	DateTime	nullable
registered_at	DateTime	default now
updated_at	DateTime	auto update

Notifications / Communication

CALL SESSIONS

Stores doctor-scheduled calls between listed patients and caregivers.

Column	Type	Key / Notes
id	UUID	PK, default uuid
caller_id	UUID	FK -> users.id, indexed
callee_id	UUID	FK -> users.id, indexed
call_type	Enum	audio video
status	Enum	idle ringing in_call ended rejected busy missed failed
offer_dlp	String(5000)	nullable
answer_dlp	String(5000)	nullable
duration_seconds	Integer	default 0
started_at	DateTime	nullable
answered_at	DateTime	nullable
ended_at	DateTime	nullable
created_at	DateTime	default now

AI / Symptom Checker

ASSESSMENT SYMPTOMS

Stores selected symptoms inside an assessment.

Column	Type	Key / Notes
id	UUID	PK, default uuid
assessment_id	UUID	FK -> assessments.id, indexed
symptom_id	String(100)	indexed
severity	Integer	expected 0-4

AI / Symptom Checker

CHAT SESSIONS

Stores chatbot-scheduled symptom-checker chat sessions.

Column	Type	Key / Notes
id	UUID	PK, default uuid
user_id	UUID	nullable FK -> users.id, indexed
assessment_id	UUID	nullable FK -> assessments.id, indexed
status	String(50)	default INITIAL
needed_from_assessment	Boolean	default false
lang	String(5)	default en
context	JSONB	default {}
created_at	DateTime	indexed, default now
updated_at	DateTime	auto update

Note: OCK / prescription / document tables are not implemented in the current backend; updated life URLs are stored in existing image_url/profile fields.

Figure 5.2: Complete database tables reference exported from the implemented backend models.

5.3 Relationships Worth Calling Out

- **One many-to-many junction.** `patient_caregiver_links` is the only many-to-many table; every other user-owned table is a straightforward one-to-many from `users`. A partial unique index (`WHERE is_primary AND is_active`) enforces at most one active primary caregiver per patient — a real database-level invariant, not just an application check.
- **The BP-triage database trace.** `assessments.source_vital_id` is a nullable FK straight to `vital_signs`: when a triage runs off a real BP reading, the resulting assessment row links directly back to the vital sign that triggered it.
- **Nullable BP by design.** `vital_signs.systolic/ diastolic` are nullable because a sensor-submitted reading can exist in a `completed_pending_bp` state (HR/SpO2 known, BP not yet confirmed) before a cuff-confirmed value arrives.
- **Two device-registry tables.** `iot_devices` is a generic registry used by the mock path, while `registered_devices` is the table actually used for BP-hardware authentication in production; they do not reference each other.

5.4 Business Rules Encoded in the Schema and Domain Layer

Table 5.2: Key business rules, as encoded in the domain layer.

Rule	Definition
BP risk tiers	Low < 90/60; Normal 90/60–120/80; Moderate 120–139/80–89; High 140–179/90–119; Critical $\geq 180/120$
Symptom urgency	Critical if any red flag severity ≥ 3 or critical vitals; High if any red flag or high vitals; Moderate if ≥ 2 symptoms at severity ≥ 2 or known hypertension + elevated vitals; else Low
Red flags	Most trigger at severity ≥ 2 ; syncope/LOC variants at any severity ≥ 1
Calibration	< 8 paired samples $\rightarrow$ median offset only; $\geq 8 \rightarrow$ least-squares linear fit; both clamp scale $\in [0.70, 1.30]$, offset $\in [-25, +25]$
Calibration drift	Flagged on sustained shift >8 mmHg (SBP) or >5 mmHg (DBP) from baseline, or age >90 days; re-notify throttled to once / 14 days
Emergency cooldown	15 min for critical, 30 min for high/low, per patient per tier
Primary caregiver	One active primary per patient, enforced at the database level

Chapter 6

Methodology

This chapter documents the two AI subsystems end to end: the data they were built from, how that data was preprocessed and represented, how the models were trained and compared, and how the trained artifacts were carried into production. Everything below is drawn from the training notebook, the training scripts, and the generated artifacts they produced.

6.1 Data Collection and Preprocessing

6.1.1 Blood-Pressure Dataset

The blood-pressure model was trained on a MIMIC-derived cuffless dataset [1]: twelve `.mat` files, where each record is a $[3, N]$ array of synchronized PPG, arterial blood pressure (ABP, the reference signal), and ECG, originally sampled at 125 Hz. The ABP channel is an *invasive arterial-line* reading — this is genuine ground truth for blood pressure, though it is arterial-line pressure, not sphygmomanometer cuff pressure, and the two are not identical in every clinical sense.

Splitting is done at the **record level**, not the window level, to prevent information leakage:

- 15% of records held out as test.
- Of the remaining 85%, another 15% held out as validation.
- An explicit no-overlap assertion between the three index sets before proceeding.

A window-level split would leak adjacent, overlapping windows from the same patient into both train and test and produce an optimistic accuracy estimate. After extraction and quality filtering, the final window counts are given in Table 6.1; the test set spans 1,631 distinct records, later reused for the per-patient calibration simulation.

Table 6.1: Final windowed dataset sizes after record-level splitting and quality filtering.

Split	Windows
Train	455,596
Validation	99,340
Test	98,079
Total	653,015

Windowing and Reference Labels

Each record is bandpass filtered, resampled from 125 Hz to 100 Hz, then cut into 8-second (800-sample) windows with 50% overlap (step size 400 samples). For each window:

- Systolic/diastolic reference values are derived from the ABP channel itself, by peak/trough detection and amplitude averaging — not from a separate cuff reading.
- Windows are rejected if peak/trough counts fall outside a plausible heart-rate range (40–180 bpm), or if the derived SBP/DBP fall outside plausible bounds (SBP 70–200 mmHg, DBP 40–130 mmHg, DBP < SBP required).
- A signal-to-noise gate (−7 dB) is applied to the PPG channel; on this clean dataset it keeps essentially all windows and acts as a safety net.

6.1.2 Preprocessing Pipeline

The same three-step pipeline is applied to every window and — importantly — the exact same functions are reused verbatim in the production inference code, so there is no drift between training and inference:

1. **Bandpass filter** — 4th-order Butterworth, 0.5–8 Hz for PPG and 0.5–40 Hz for ECG (zero-phase `filtfilt`).
2. **Z-score normalization** — $(x - \mu)/(\sigma + 10^{-8})$, per window, per channel.
3. **Feature extraction** — 21 hand-crafted features (below), also z-scored using a scaler fit only on the training set.

Figures 6.1–6.2 show this pipeline applied to synthetic signals shaped like real MAX30102/AD8232 output, demonstrating that raw sensor units (ADC counts, with a $\sim 150,000$ DC offset for the PPG photodiode and ~ 512 for the single-ended ECG front-end) collapse onto the same normalized scale the model was trained on — a deliberate check performed before ever wiring up real hardware.

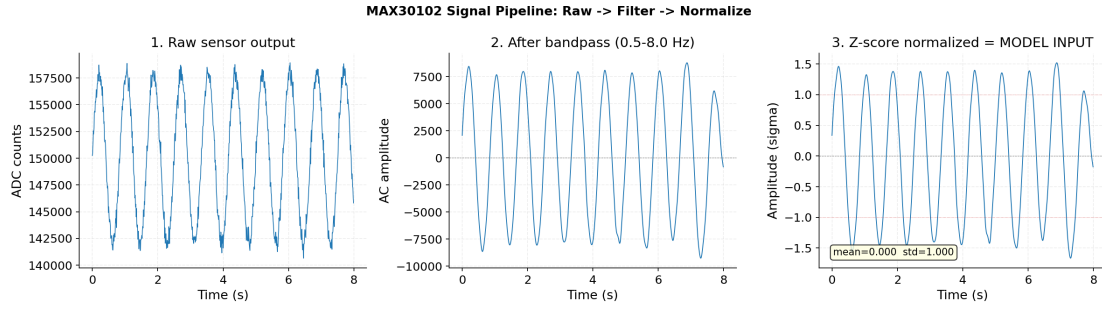


Figure 6.1: PPG (MAX30102) preprocessing pipeline: raw ADC counts → bandpass → z-score normalized.

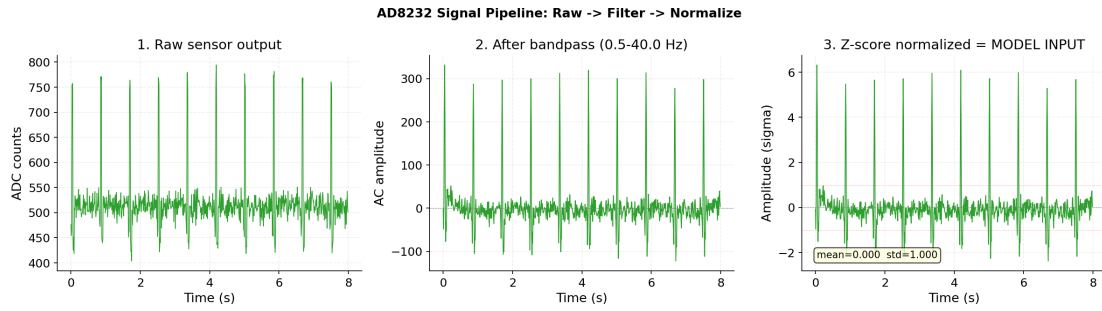


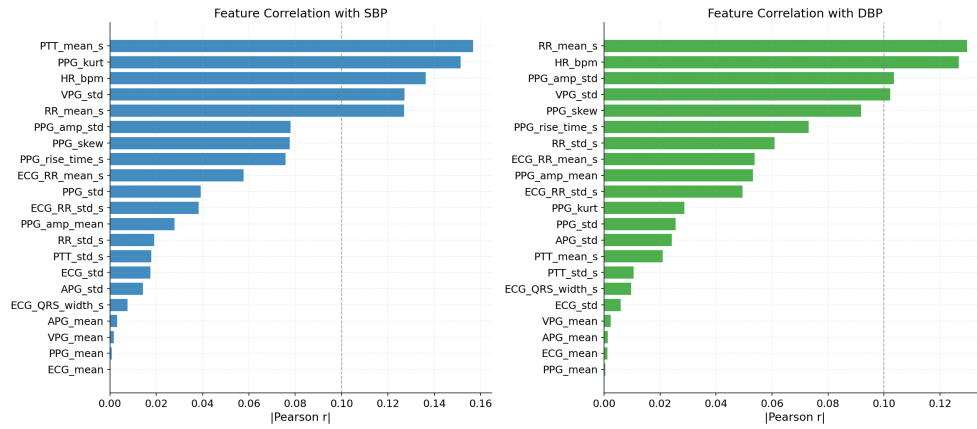
Figure 6.2: ECG (AD8232) preprocessing pipeline: raw single-ended output → bandpass → z-score normalized.

6.1.3 Feature Engineering

Alongside the raw waveform, a 21-dimensional hand-crafted feature vector is computed per window and fed into a small dense side-branch (Table 6.2). Pulse transit time (PTT) is the feature most grounded in physiology — it is the classical basis for cuffless BP estimation [2]. The notebook’s correlation analysis (Figure 6.3) confirms that PTT and heart rate carry the strongest linear correlation with SBP/DBP among the 21 features, though none individually correlates strongly enough to justify a pure feature-based (non-waveform) model.

Table 6.2: The 21 hand-crafted features fed to the model’s dense side-branch.

Index	Feature(s)	Description
0–2	HR_bpm, RR_mean_s, RR_std_s	Heart rate and RR-interval statistics from PPG peaks
3–4	PTT_mean_s, PTT_std_s	Pulse transit time: PPG peak minus preceding ECG R-peak
5	PPG_rise_time_s	Time from preceding trough to peak
6–7	PPG_amp_mean, PPG_amp_std	Pulse amplitude (peak – preceding trough)
8–11	PPG_mean/std/skew/kurt	Basic PPG waveform statistics
12–13	VPG_mean, VPG_std	First derivative of PPG (velocity plethysmogram)
14–15	APG_mean, APG_std	Second derivative of PPG (acceleration plethysmogram)
16–17	ECG_RR_mean_s, ECG_RR_std_s	RR interval from ECG R-peaks
18	ECG_QRS_width_s	Approximate QRS width around each R-peak
19–20	ECG_mean, ECG_std	Basic ECG waveform statistics

**Figure 6.3:** Correlation of the 21 hand-crafted features with SBP/DBP. PTT and heart rate dominate, but no single feature is strong enough on its own.

6.1.4 Class Imbalance: Sample Weights, Not Downsampling

Windows are labeled into four BP categories (Hypotension, Normal, Elevated, Hypertension). These are heavily imbalanced (Figure 6.4). An earlier internal attempt (“v10”) forced class balance by discarding windows down to 50K per class, which threw away most of the real dataset and made the model *worse* (SBP MAE 11.47 versus an earlier unbalanced 9.13). The final approach instead computes a per-window sample weight from inverse class frequency, clipped to a maximum $5\times$ ratio (Figure 6.5), and keeps every real window.

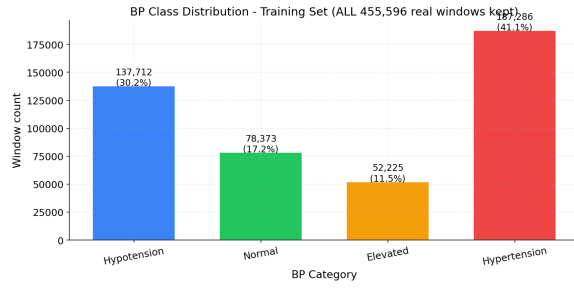


Figure 6.4: BP-category distribution: heavily imbalanced toward the Hypertension and Hypotension extremes.

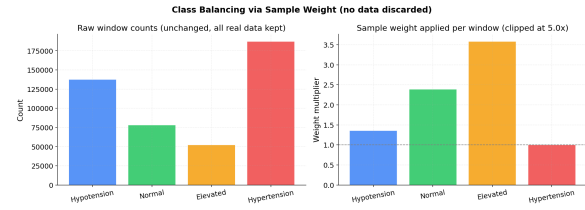


Figure 6.5: Inverse-frequency sample weights, clipped to a $5\times$ maximum ratio.

6.1.5 Data Augmentation

A modest synthetic augmentation (15% of the real training-set size, versus 40% in the earlier “v10” attempt) is layered on top of the full real dataset, using three sensor-realistic perturbations applied together per augmented sample: additive white Gaussian noise (target SNR 25 dB), slow baseline wander (0.05–0.3 Hz sinusoid, random phase/amplitude), and random amplitude scaling ($0.8\text{--}1.2\times$ for PPG, $0.85\text{--}1.15\times$ for ECG). This adds 68,339 synthetic windows on top of the 455,596 real ones, for an effective training set of 523,935 windows, shuffled so real and synthetic windows interleave.

6.1.6 Symptom-Checker Dataset

The structured (v2) symptom model’s training data is explicitly **synthetic**, generated from a taxonomy of diseases, symptoms, and categories rather than collected from real patients:

- Each disease’s related-symptom list becomes a symptom-inclusion probability profile (symptoms listed earlier get higher probability, from 0.85 down to a 0.35 floor).
- Each synthetic case independently samples severity, duration, age group, and optional risk factors/vitals around that profile.
- A target of 300 cases per disease is generated, split into 9,840 train / 2,460 test at training time across 40 disease classes.
- **Urgency labels are not hand-assigned** — every generated case is run through the same production rule engine that judges real assessments, so the dataset can never encode an urgency value that disagrees with the live logic.
- Targeted differentiation was applied to two commonly-confused clusters: Common Cold / Influenza / COVID-19 (separated by duration, severity, and a boosted probability of loss of taste/smell for COVID-19), and Hypertension / Migraine / Stroke (whose symptom sets already do not overlap in the taxonomy).

The resulting per-case representation is a 232-dimension feature vector, a single `build_features()` function shared verbatim between training and production inference (Table 6.3).

Table 6.3: Layout of the 232-dimension structured symptom feature vector.

Segment	Size	Content
Symptom presence	N_{sym}	Multi-hot: 1 if the symptom was selected
Symptom severity	N_{sym}	0 if absent, else the reported severity
Duration bucket	4	One-hot: <1, 1–3, 4–7, >7 days
Age group	3	One-hot: child / adult / elderly
Risk factors	5	Multi-hot over a comorbidity pool (hypertension, T2 diabetes, asthma, hypothyroidism, CKD)
Vitals	4	Normalized systolic/diastolic/heart rate (clipped) + a <code>vitals_present</code> flag

6.2 Model Development

6.2.1 Blood-Pressure Models

Ten models were trained and evaluated on the identical test set, all using the same 21 features (classical) or the same waveform + feature combination (deep learning), and all trained with the same sample weights:

- **Classical ML (6):** Ridge, ElasticNet, Random Forest, XGBoost, LightGBM, CatBoost — each predicting SBP/DBP jointly.
- **Deep learning (4):** 1D-CNN, BiLSTM, TCN, and the team’s proposed TCN+BiLSTM+Attention model. All four share a two-branch design: a waveform encoder consuming the raw (800, 2) PPG+ECG tensor, and a small dense branch consuming the 21 features, concatenated before a shared prediction head (Table 6.4).

Table 6.4: The four deep-learning architectures and their parameter counts.

Model	Encoder	Params
1D-CNN	4-stage Conv1D + MaxPooling, global average pool	485,154
BiLSTM	2 Conv+Pool stages (800→50 steps) feeding a bidirectional LSTM	371,234
TCN	4-stage dilated causal Conv1D residual blocks (dilations 1/2/4/8)	1,017,634
TCN+BiLSTM+Attn (ours)	TCN → BiLSTM → 4-head self-attention → residual + LayerNorm	1,544,610

The conceptual two-branch design and the best model’s exact Keras layer graph are shown in Figures 6.6–6.7, and the exact auto-generated layer graphs of the other three

architectures (BiLSTM, TCN, and the team’s full TCN+BiLSTM+Attention model) are shown together in Figure 6.8. Each architecture’s graph was produced directly by Keras `plot_model` on the trained model, so these are the real, verifiable network structures, not redrawn approximations. The BiLSTM architecture is worth calling out: an earlier version ran the LSTM directly over all 800 raw timesteps and was the slowest model in the ablation (64 ms/step); the current version downsamples to 50 steps with two Conv+Pool stages first — a documented performance fix, not an incidental choice.

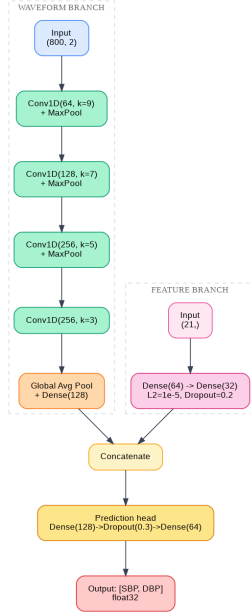


Figure 6.6: Conceptual two-branch design: a waveform encoder and a dense feature branch, concatenated before a shared prediction head.

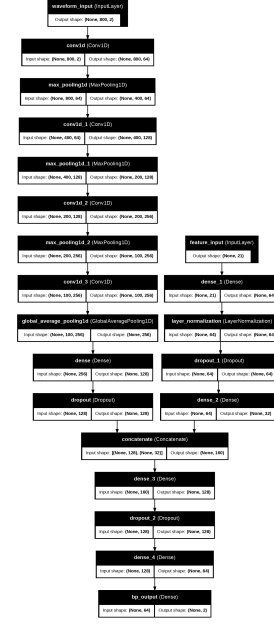


Figure 6.7: Exact Keras layer graph of the best-performing 1D-CNN model (auto-generated by `plot_model`).

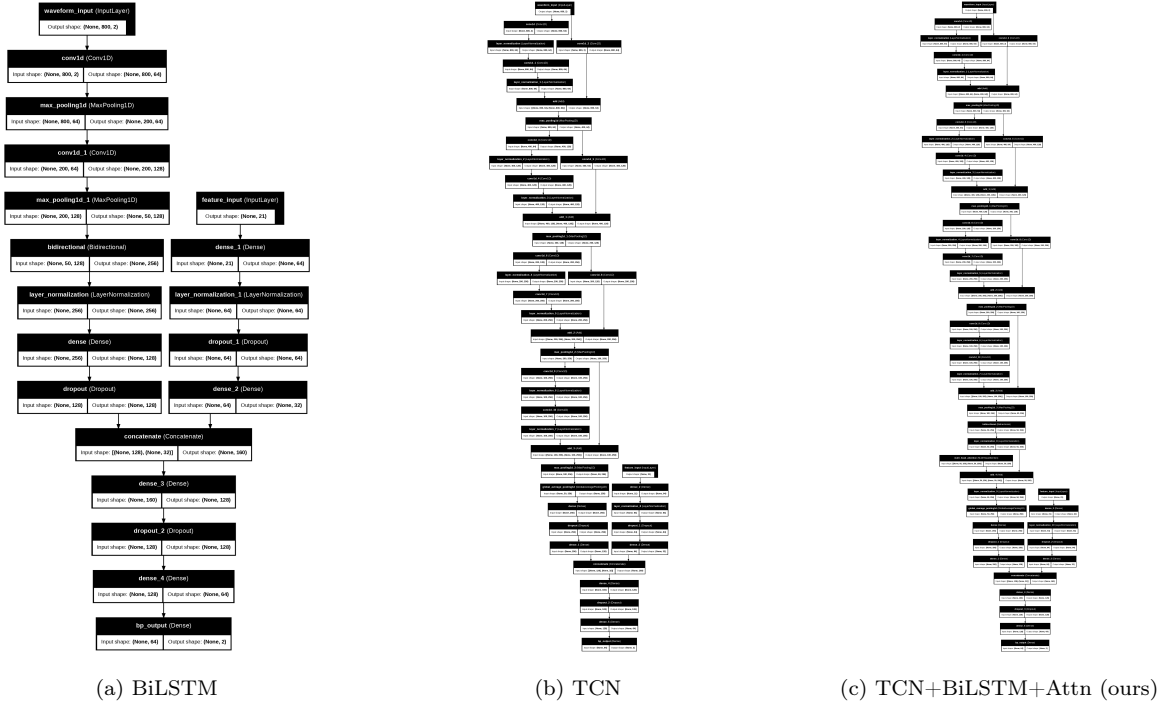


Figure 6.8: Exact Keras layer graphs of the other three deep-learning architectures. The increasing depth from (a) to (c) reflects the rising parameter counts in Table 6.4 — and, as the results show, that added depth did not translate into better accuracy.

Regularization: a Documented Course-Correction

The notebook explicitly documents why the current hyperparameters differ from an earlier attempt (“v4”): v4 used *no* L2 regularization, producing an 86% train/validation MAE gap (severe overfitting). The current configuration uses light L2 (10^{-5}) and 0.3 dropout, bringing the healthiest models’ overfitting gap down substantially (Figure 6.9). A dead decoder branch (from an abandoned reconstruction loss that never moved during training) was also removed. Training uses Huber loss (robust to noisy windows), Adam with gradient clipping, learning-rate reduction on plateau, and early stopping on validation MAE with best-weight restoration, on a dual-T4 Kaggle session with mixed precision.

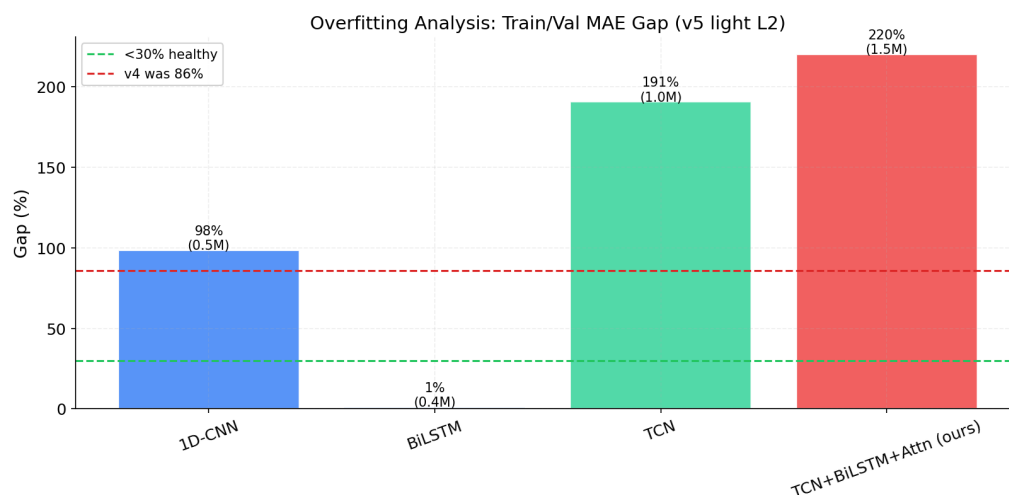


Figure 6.9: Train-versus-validation MAE gap across models after the regularization course-correction.

Fault Tolerance and Reproducibility

Every expensive stage — window extraction, feature engineering, and each model’s training — checkpoints to disk. If the session is interrupted (a real risk on free Kaggle GPU quotas) and restarted, completed stages are detected from cached files and skipped. Each model is wrapped in its own try/except, so one model crashing does not take down the run. This resumable design (Figure 6.10) is what made training practical across multiple sessions on a shared, quota-limited GPU.

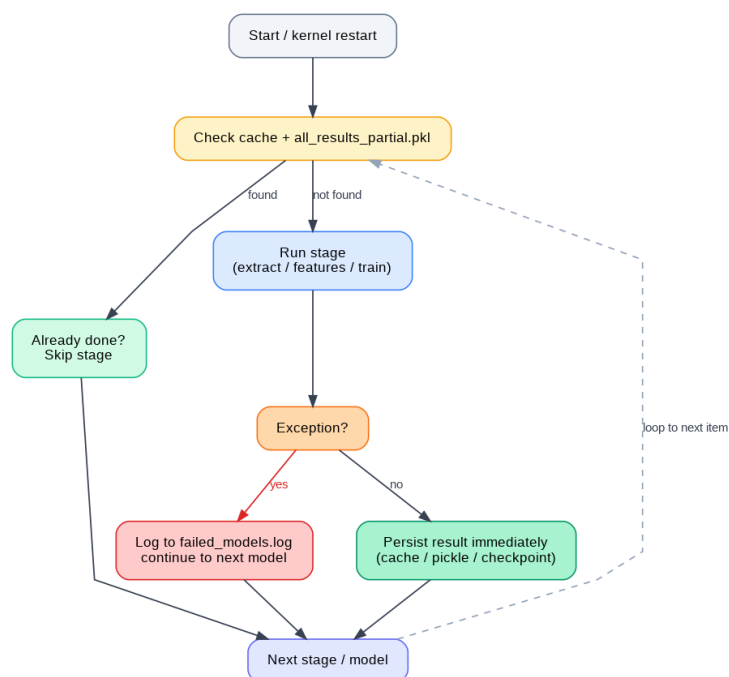


Figure 6.10: Fault-tolerant, resumable training workflow: cache-and-resume at every stage, with per-model isolation.

6.2.2 Symptom-Checker Model

The production candidate is a LightGBM classifier [5] with a multiclass objective, 300 estimators, learning rate 0.05, over the 40 disease classes and the 232-feature vector. Two simpler baselines (Logistic Regression, Random Forest) were trained on the identical feature matrix and split specifically for comparison — a decision that turns out to matter (Section 7.1.2).

6.3 Implementation

The trained BP model’s exact preprocessing and inference logic (bandpass filter → z-score normalize → feature extraction → model call → inverse-transform to mmHg) is implemented once in the training notebook and reimplemented in the backend’s inference service using the identical function bodies. The three exported artifacts — `bp_model_final.keras`, `scaler_features.pkl`, and `scaler_targets.pkl` — are copied verbatim into the backend. On top of the notebook template, the backend service adds:

- **Graceful startup** — if the model files are missing, the service reports `model_not_ready` instead of crashing the API.
- **Input validation** — exact window length and all-finite checks before inference.
- **Physiological clamping** — output clamped to SBP 50–300, DBP 30–200, HR 30–220 as a final safety net before any number reaches a patient screen.

The equivalent guarantee exists on the symptom-checker side: the 232-feature builder is written once and imported directly by the backend at inference time (via a runtime cross-directory import), rather than reimplemented, so the training and inference feature layouts can never silently diverge. Both models are loaded in-process by the FastAPI backend rather than served by a separate inference process, each with an explicit “not ready” fallback rather than a crash if artifacts are missing at startup.

Chapter 7

Results and Discussion

7.1 Model Performance

7.1.1 Blood-Pressure Results

Table 7.1 gives the final test-set metrics for all ten models, sorted by SBP MAE. Figures 7.1–7.2 visualize the same comparison against the AAMI limit and the internal “v4” baseline.

Table 7.1: Final test-set results for all ten blood-pressure models, sorted by SBP MAE. Lower MAE is better; BHS grades run A (best) to D (worst).

Model	SBP MAE	SBP BHS	DBP MAE	DBP BHS
1D-CNN (best)	9.08	D	4.91	B
TCN	9.79	D	5.31	B
TCN+BiLSTM+Attn (ours)	10.17	D	5.63	B
BiLSTM	14.34	D	7.28	C
XGBoost	14.71	D	7.45	C
CatBoost	15.13	D	7.58	C
LightGBM	15.16	D	7.59	C
Random Forest	15.80	D	7.83	C
Ridge	18.03	D	8.59	D
ElasticNet	18.37	D	8.73	D

Against the reference clinical standards:

- **AAMI SP10** (requires $\text{MAE} \leq 5 \text{ mmHg}$ and $\text{SD} \leq 8 \text{ mmHg}$): every model *fails* this for SBP. The best model (1D-CNN) *passes* it for DBP (MAE 4.91, SD 7.77).
- **BHS protocol** [8] grades A–D by the percentage of errors within 5, 10, and 15 mmHg. The best SBP result is Grade D; DBP reaches Grade B.

The simplest deep-learning architecture — the plain 1D-CNN — was the best performer overall, beating both the TCN and the more elaborate TCN+BiLSTM+Attention model the team specifically designed for this problem. All three deep-learning waveform

models comfortably outperformed every classical feature-only model, consistent with the raw waveform carrying information the 21 hand-crafted features alone do not fully capture. The result is documented as-is; the notebook does not attempt to explain away why the attention model underperformed, and neither does this document.

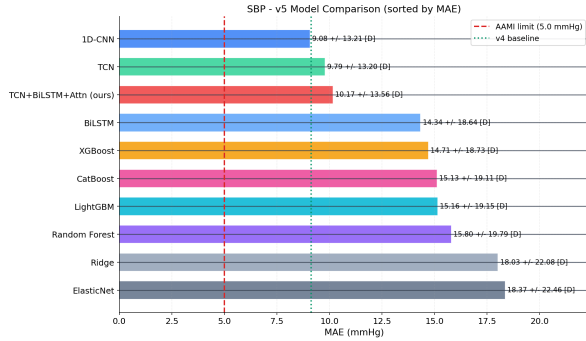


Figure 7.1: SBP MAE across all ten models against the AAMI limit (5 mmHg) and the v4 baseline.

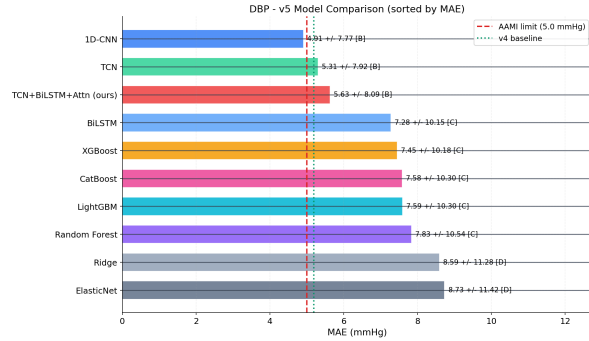


Figure 7.2: DBP MAE across all ten models; the best model clears the AAMI limit for diastolic pressure.

Clinical Evaluation of the Best Model

The clinical-agreement figures for the best model (1D-CNN) are shown in Figures 7.3–7.6. The scatter plots (Figure 7.3) show true-versus-predicted values colored by absolute error with ± 5 mmHg reference bands; the Bland–Altman plots (Figure 7.4) [7] show mean bias and 95% limits of agreement; and the per-class MAE breakdown (Figure 7.6) reveals whether error is evenly spread or concentrated in the rarer BP categories that sample weighting was designed to protect.

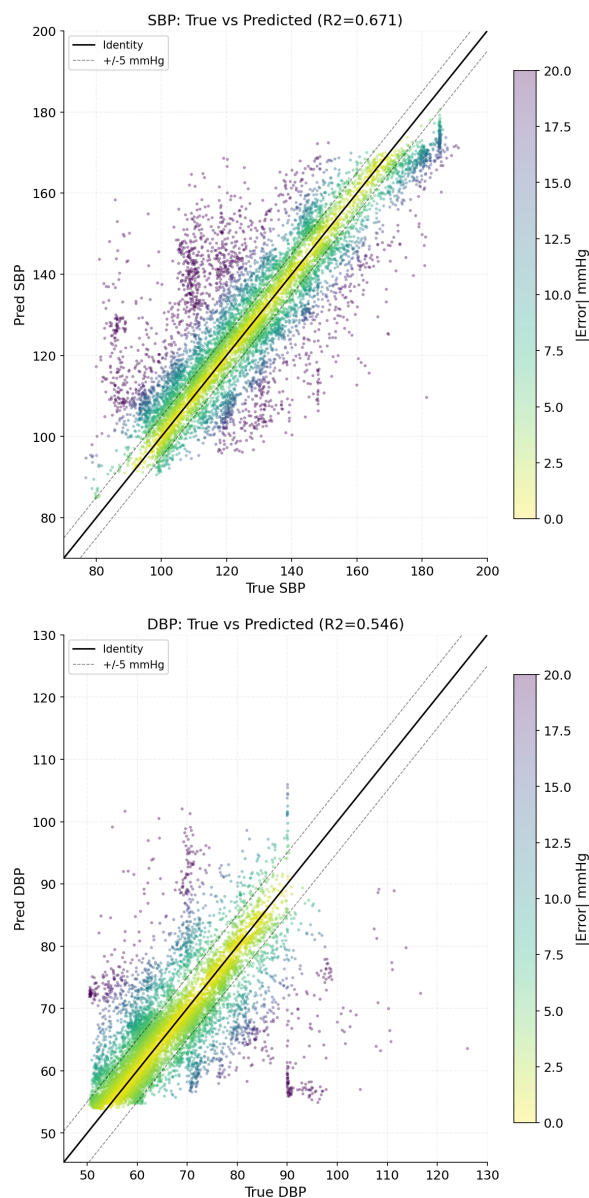


Figure 7.3: True vs. predicted SBP (top) and DBP (bottom), colored by absolute error.

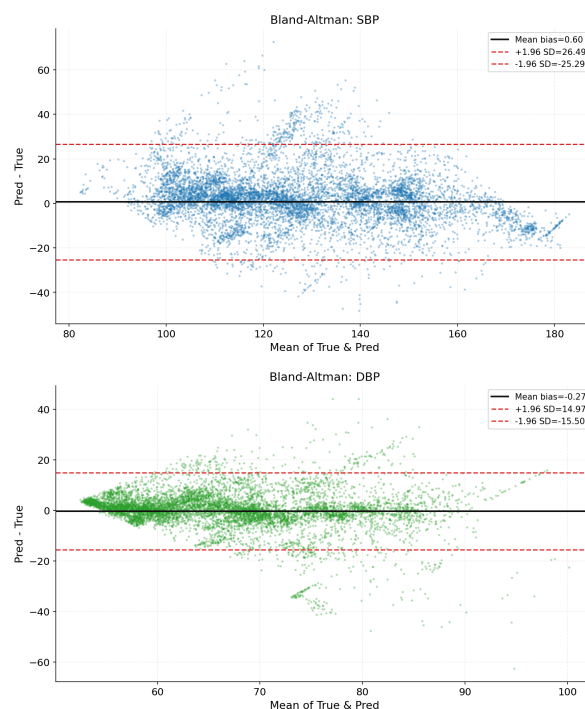


Figure 7.4: Bland-Altman agreement for SBP (top) and DBP (bottom): mean bias $\pm$ 1.96 SD limits of agreement.

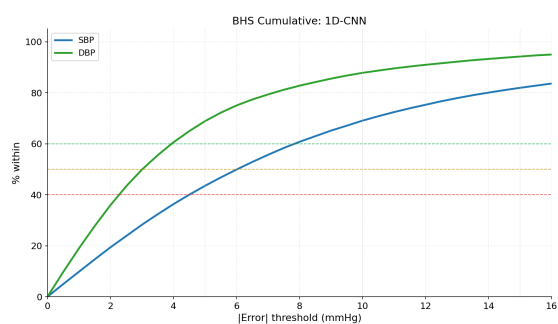


Figure 7.5: Cumulative error-threshold curve for the best model against the BHS A/B/C grade bands.

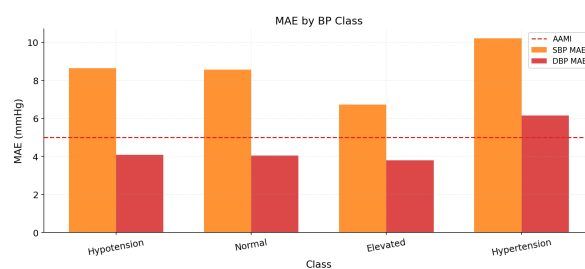


Figure 7.6: Per-class MAE across the four BP categories: error is not uniform across the pooled AAMI/BHS headline.

For context, the notebook tracks a “v4 baseline” (SBP MAE 9.13, DBP MAE 5.19) from an earlier, larger, unregularized model. The current best model (SBP MAE 9.08) is a small improvement on SBP and a slightly larger one on DBP (4.91 vs. 5.19) — modest gains achieved with better-controlled overfitting rather than a fundamentally different result. Figure 7.7 summarizes the rationale behind the final ablation choices.

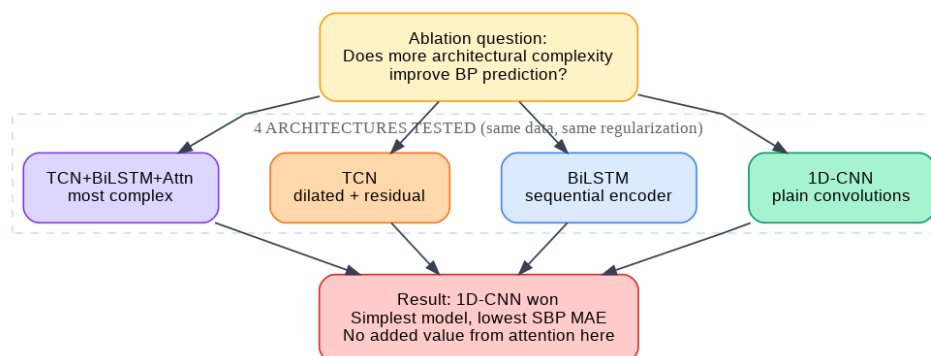


Figure 7.7: Ablation rationale: how the final configuration was reached from the earlier over-parameterized and over-balanced attempts.

7.1.2 Symptom-Checker Results

From the model’s own generated metadata and evaluation report:

- **Top-1 accuracy: 0.8581**
- **Top-3 accuracy: 0.9549**

The report explicitly disowns an older 0.9034 accuracy figure sometimes quoted for the v1 model as a fair comparison point, because that number was measured on the same narrow synthetic method used to train it, with no held-out real-world distribution.

The Confusable-Pair Check Works

The three-way Common Cold / Influenza / COVID-19 split and the Hypertension / Migraine / Stroke split are both resolved with *zero* cross-confusion in the held-out test set — the targeted differentiation strategy achieved exactly what it set out to do.

The Ten Worst Classes Are Cardiac

The ten worst-performing classes are almost entirely cardiac subtypes (Table 7.2). Because cardiovascular disease is the project’s core clinical focus, this is a meaningful, practically relevant weakness, not a peripheral one: the model is weakest exactly where the product needs it to be strong. These diseases share heavily overlapping symptom sets in the taxonomy (fatigue, palpitations, chest pain, dyspnea, edema), and the synthetic generator did not apply the same targeted differentiation to these cardiac pairs that it applied to the respiratory and hypertension/neurological triads.

Table 7.2: The ten lowest per-class top-1 accuracies — almost all cardiac subtypes.

Disease	Top-1 accuracy
Myocarditis	0.350
Hypertrophic cardiomyopathy	0.400
Coronary artery disease	0.517
Congestive heart failure	0.600
Pericarditis	0.600
Ventricular tachycardia	0.650
Heart failure	0.683
Pulmonary hypertension	0.683
Stable angina	0.700
Aortic stenosis	0.750

The Production Model Is Not the Best-Scoring Model

A more fundamental, easy-to-miss finding: the baseline comparison (Table 7.3), trained on the identical feature matrix and split, shows plain Logistic Regression scoring **0.8703** and Random Forest **0.8663** — both higher than LightGBM’s 0.8581. The model chosen for production is not the one that scored highest on this exact dataset. This mirrors the BP result, where the simplest deep-learning model also beat a more sophisticated one, and is reported here with the same lack of spin.

Table 7.3: Symptom-checker model comparison on the identical structured dataset.

Model	Top-1 accuracy	Status
Logistic Regression	0.8703	baseline (for comparison)
Random Forest	0.8663	baseline (for comparison)
LightGBM	0.8581	deployed to production

7.2 Interpretation of Results

Two of this project’s clearest results do not flatter its own preferred architectures, and both are reported here rather than adjusted after the fact. First, greater model sophistication did not translate into better blood-pressure accuracy: the combined TCN+BiLSTM+attention model underperformed a plain 1D-CNN, most plausibly because an 8-second, fixed-length single window does not have much long-range temporal structure for a bidirectional recurrent layer or self-attention to exploit. Second, the same pattern recurs independently in the symptom checker, where LightGBM was outperformed by a plain Logistic Regression baseline on the identical feature set. Encountering this pattern in two separate

subsystems is itself the project’s most transferable lesson: architectural complexity must earn its place against a simple baseline that is actually run, not assumed away.

Both the cardiac-subtype weakness (symptom checker) and the SBP AAMI shortfall (BP model) point to the same underlying theme: the project’s two hardest, most clinically relevant sub-problems are exactly the areas where the current models are weakest — a substantive, not cosmetic, limitation of the current system.

Is “Best Model” the Same Claim as “Clinically Acceptable”?

It is worth being explicit about a distinction this document has used throughout but not yet named directly, because it is exactly the kind of question a careful reviewer should ask: the 1D-CNN is called the *best* blood-pressure model, and yet the same section states it *fails* the AAMI SP10 standard [9] for systolic pressure. Both statements are true at once because they answer two different questions. “Best” is a *comparative* claim — among the ten models actually trained and evaluated on the identical test set, the 1D-CNN has the lowest SBP and DBP error of all of them. “Passes AAMI” is an *absolute* clinical-acceptance threshold, unrelated to how the model compares to any alternative — it asks only whether the error is small enough ($\text{MAE} \leq 5 \text{ mmHg}$, $\text{SD} \leq 8 \text{ mmHg}$) for a device to be considered clinically interchangeable with a reference sphygmomanometer. A model can legitimately be the best of a field and still not clear an absolute bar the entire field misses.

Given that framing, choosing the 1D-CNN as the production model is, in this document’s assessment, still the correct engineering decision, for three concrete reasons rather than as a default: first, *no* model in the ten-model comparison passes AAMI for SBP, so discarding the 1D-CNN in favor of any alternative would not produce a passing model either — it would only produce a *worse* one on both SBP and DBP simultaneously; second, the same model *does* clear AAMI for DBP, so shipping it delivers one clinically validated number and one indicative number, rather than two indicative numbers; third, the system does not quietly present the failing number as if it had passed — SBP is treated as a trend indicator throughout this documentation and the product’s own safety design (physiological clamping, per-patient calibration, and the `COMPLETED_PENDING_BP` state that withholds an unconfirmed number rather than showing it) rather than as a diagnostic-grade reading. What would genuinely be the wrong call is not shipping this model — it is shipping it *while presenting SBP as if it had met the same clinical bar as DBP*. This document deliberately does not do that, in the abstract, in this chapter, or in the conclusions (Chapter 10), and that separation is the actual safeguard, not the model’s accuracy alone. The honest summary a reviewer should take away is: the best available model was chosen, its one real shortfall is named precisely rather than hidden, and the product treats that shortfall as a constraint on how the number is used, not as a defect to be quietly rounded away.

7.3 Calibration Simulation

The notebook simulates a specific product decision: would per-patient linear calibration actually help on this exact test set? The corresponding product code already exists as

a working implementation wired into the BP submission endpoint, so the simulation is best read as a validation exercise for the general idea rather than an approximation the product still has to catch up to.

Method. For every test-set patient with at least 4 windows, the first 3 windows (in time order) are treated as an oracle calibration set — their true BP values stand in for perfect cuff readings. A scale/offset is fit by least squares from those 3 points and applied to that patient’s remaining windows; metrics are recomputed across all patients’ remaining windows pooled together.

Result. As Table 7.4 shows, calibration made *both* SBP and DBP *worse*, and DBP specifically regressed from an AAMI pass to a fail (Figure 7.8).

Table 7.4: Oracle calibration simulation: calibration degraded accuracy on this test set.

Metric	SBP raw	SBP cal.	DBP raw	DBP cal.
MAE (mmHg)	9.04	14.52	4.89	6.99
SD (mmHg)	13.15	16.20	7.74	8.45
AAMI	FAIL	FAIL	PASS	FAIL
BHS	D	D	B	C

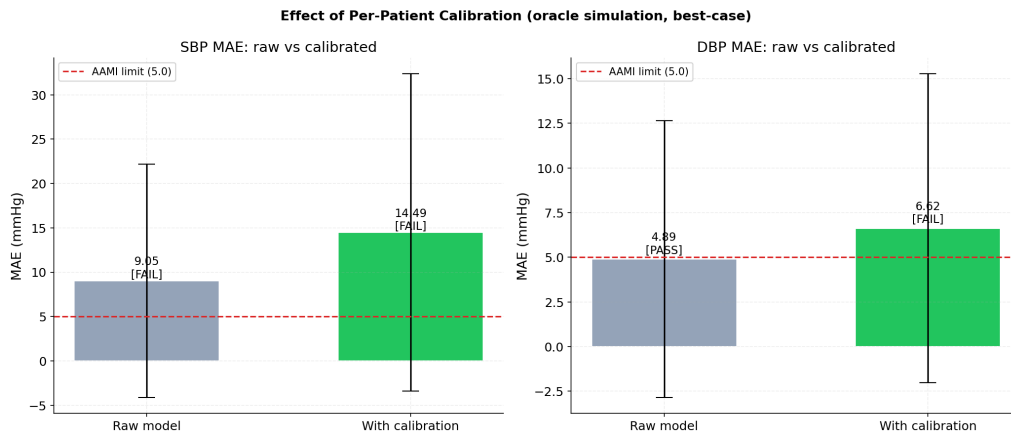


Figure 7.8: Before/after the oracle calibration simulation. The negative result is an artifact of the simulation’s sampling, not a verdict on the deployed service.

The most likely explanation is how the calibration points were chosen: the three “calibration” windows are the first three 8-second, 50%-overlapping windows of a single continuous ICU recording, not three independent measurements at meaningfully different times. The deployed calibration service accumulates its samples the way the real product measures BP — one submitted reading at a time, over however many days it takes to reach its 8-sample linear-fit threshold — a fundamentally different sampling process. This project’s interpretation is therefore that the simulation does not faithfully represent the deployed service, and its negative result should be read as evidence that the simulation’s methodology needs correcting, not as evidence against the calibration feature itself, which remains unvalidated either way pending real, time-separated data.

Chapter 8

Illustrative Examples and Feature Flows

This chapter traces the system’s main features end to end through the actual implementation, using sequence and state diagrams drawn directly from the code paths they describe.

8.1 Blood-Pressure Measurement Flow

A complete guided measurement runs as follows. A scheduled reminder opens the in-app measurement guide. The app polls live sensor status every two seconds while the patient attaches the ECG leads and places a finger on the PPG sensor; the firmware itself gates sample collection on both signals being present. Once an 8-second, 800-sample window completes, it is uploaded to the backend, which runs the model, applies the patient’s current calibration (or withholds a plain, uncalibrated number if calibration is not yet trustworthy), computes a risk tier, persists the reading, and — if the tier warrants — sends a rate-limited emergency push to every linked caregiver. Figure 8.1 shows this as a sequence diagram.

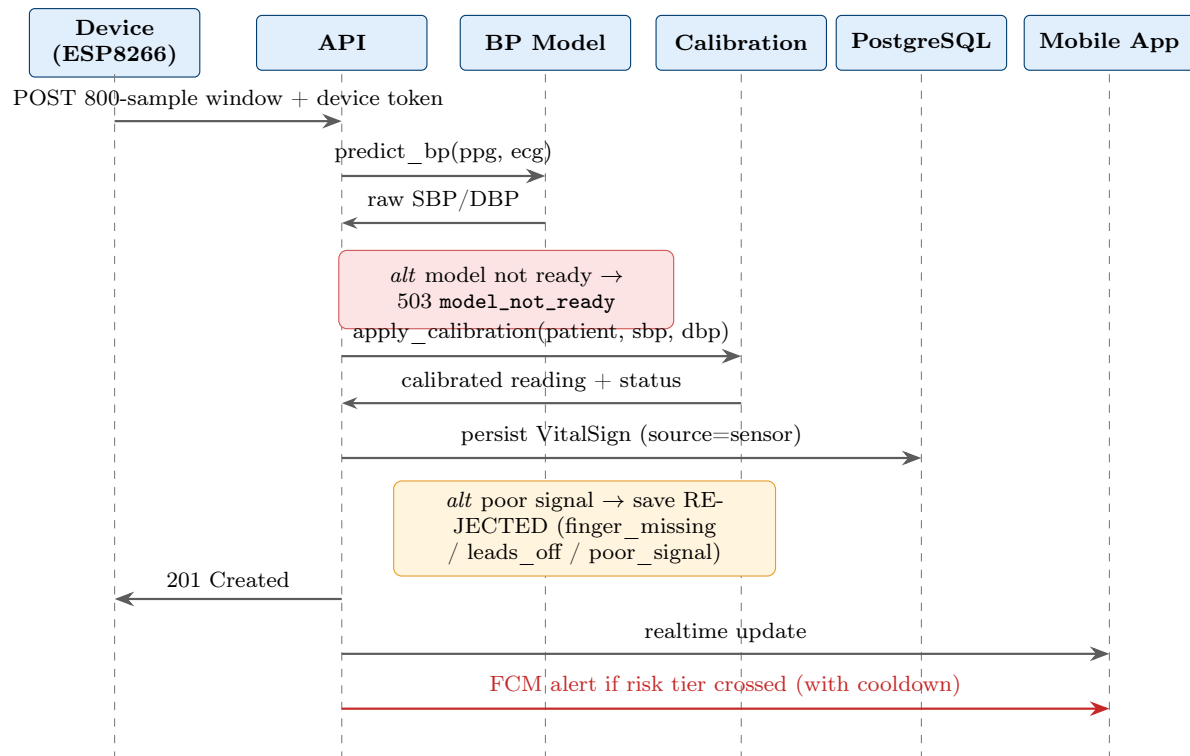


Figure 8.1: Blood-pressure measurement sequence, from the device window upload through model inference, calibration, persistence, and conditional alerting.

8.1.1 The Post-Inference State Machine

The part a patient actually experiences is a real state machine, not just “save it and show it” (Figure 8.2). The `COMPLETED_PENDING_BP` state is the detail most easily missed: until a patient has enough calibration history for the fit to be trusted (8+ paired samples, or at least a non-stale, non-clamped fit), the backend deliberately withholds the numeric SBP/DBP and requires a manual cuff confirmation to close the loop — which is also how new calibration samples are collected in the first place.

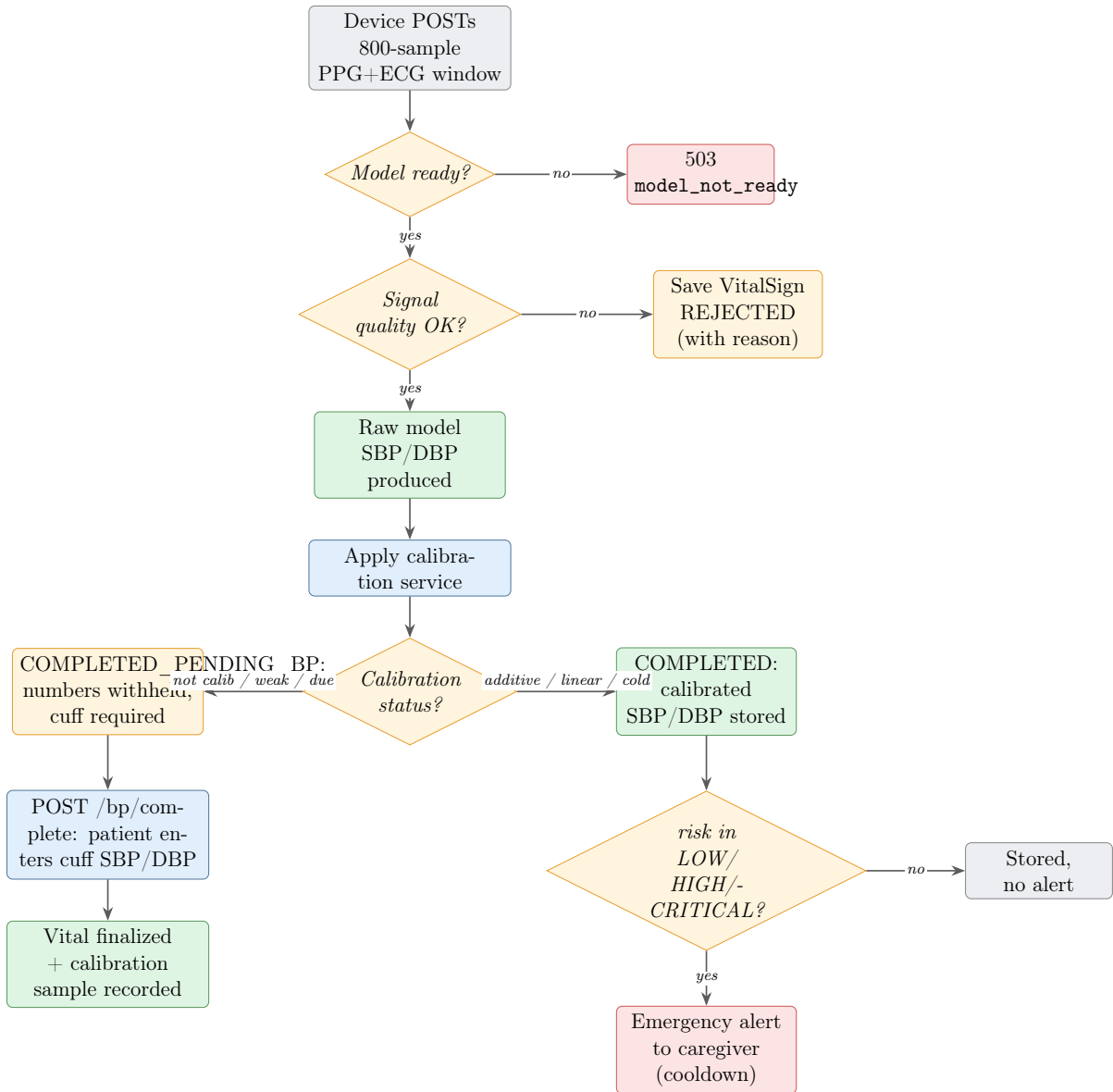


Figure 8.2: The blood-pressure submission state machine, traced from the submission endpoint. The pending-BP branch is what generates the paired samples the calibration fit later depends on.

8.2 Calibration and Drift Decision Flow

The calibration service fits $adjusted = scale \times model_output + offset$ per patient, upgrading its own fit quality as more samples accumulate, and a separate scheduled job watches for drift (Figure 8.3). Under 8 paired samples it uses a robust median-only additive offset; at 8 or more it upgrades to a least-squares linear fit. Both paths clamp scale to $[0.70, 1.30]$ and offset to $[-25, +25]$ mmHg.

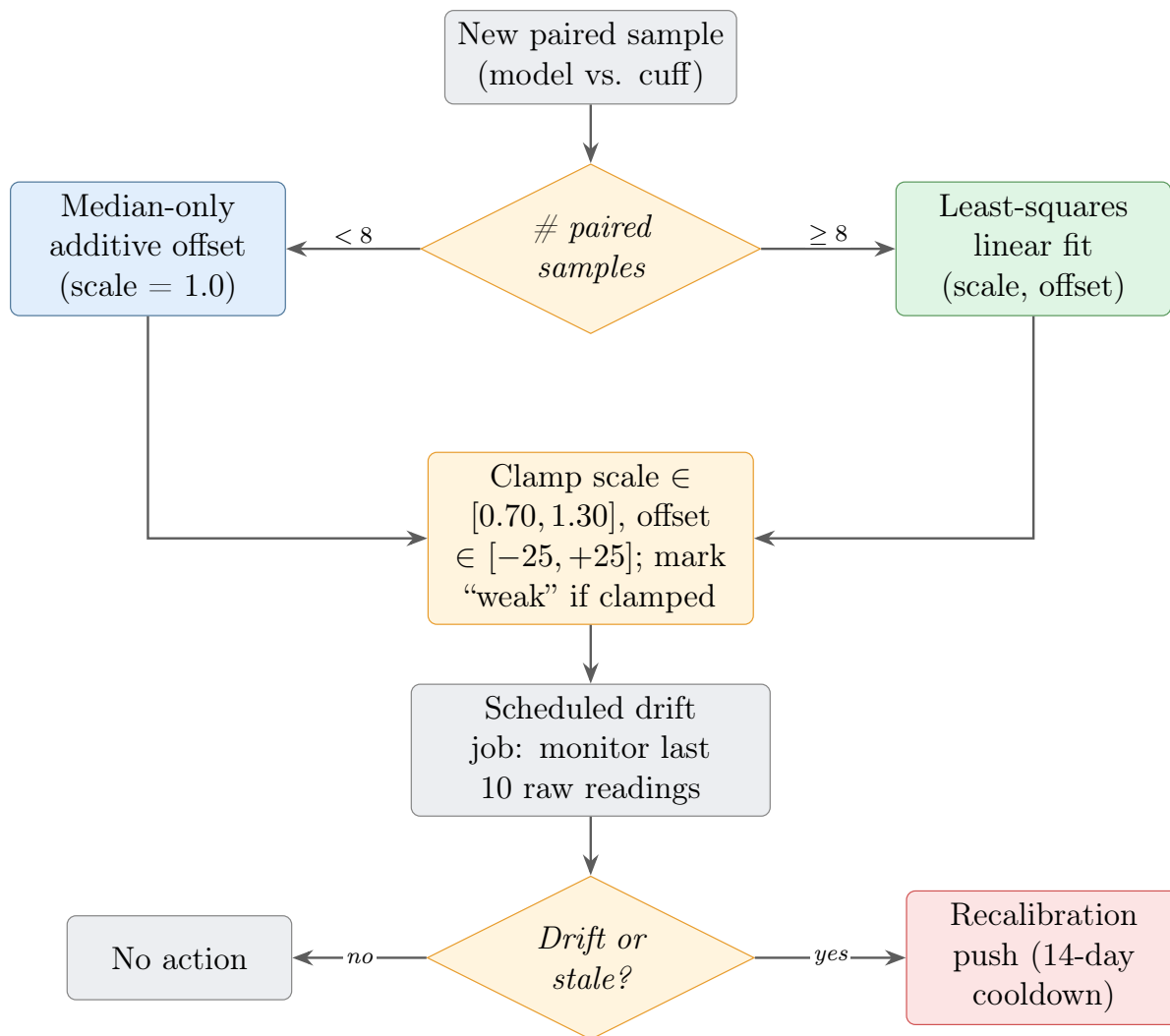


Figure 8.3: Per-patient calibration fit and the scheduled drift/staleness job. Drift is flagged on a sustained shift >8 mmHg (SBP) or >5 mmHg (DBP) from baseline, or after 90 days.

8.3 Medication Reminder Flow

When a medication alarm fires, the scheduler collects the drawers assigned to medications due at that time, optionally activates the smart box, and sends a notification (Figure 8.4). This call is deliberately fail-soft: an unreachable medicine box is logged and swallowed, never raised, so the alarm job always completes and the patient still gets the phone notification even if the physical box never lights up.

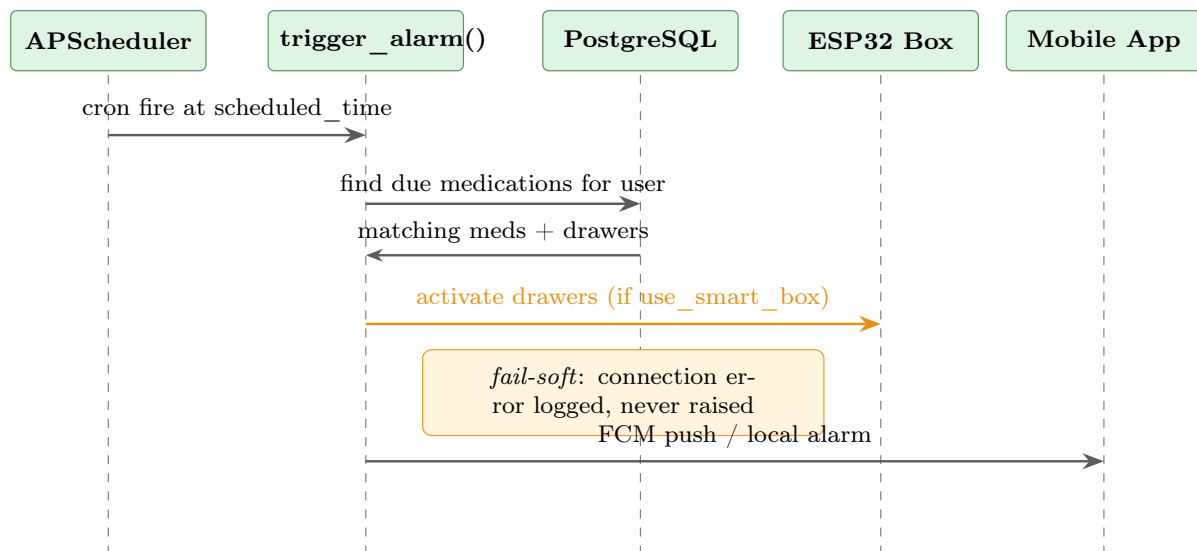


Figure 8.4: Medication reminder sequence: the smart-box activation is fail-soft, so the phone notification is delivered regardless of hardware reachability.

The complete decision tree — single vs. grouped alarms, the smart-box check, and the user’s “taken / snooze” response — is shown in Figure 8.5.

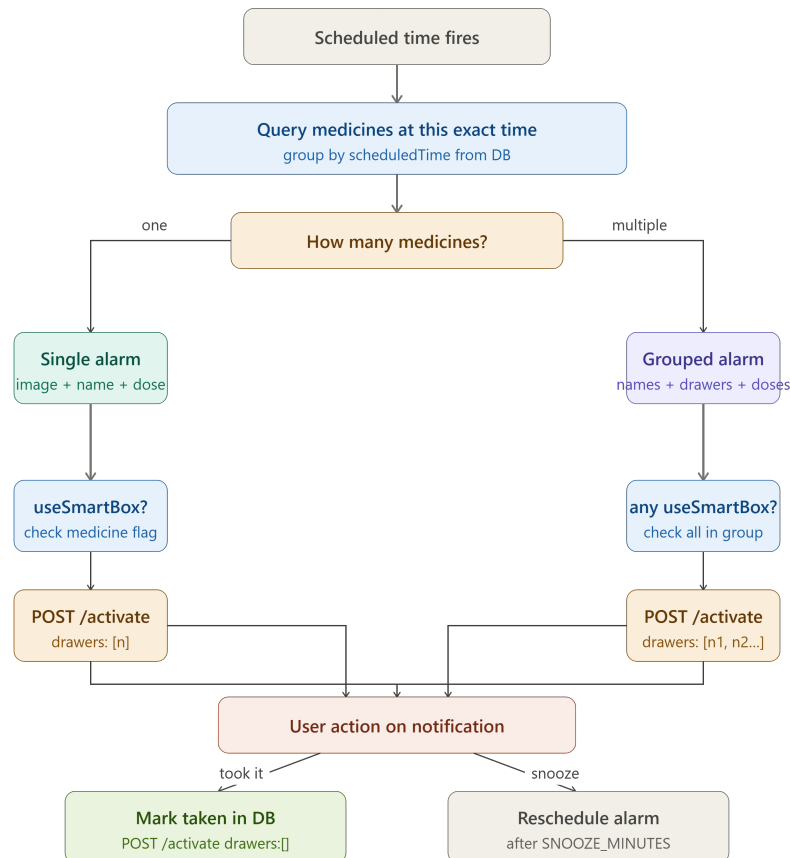


Figure 8.5: Medication alarm decision tree, including grouping of multiple medications due at the same time into one drawer-activation call.

8.4 Symptom-Triage Flow

The symptom checker exists in two parallel implementations, both reachable from the running app (Figure 8.6). The legacy v1 flow is a free-text / flat-symptom classifier with no urgency logic; the structured v2 flow is a five-step wizard whose result is jointly determined by the LightGBM classifier (for the disease prediction) and the rule engine (for red flags and urgency), with the rule engine able to escalate but never suppress urgency.

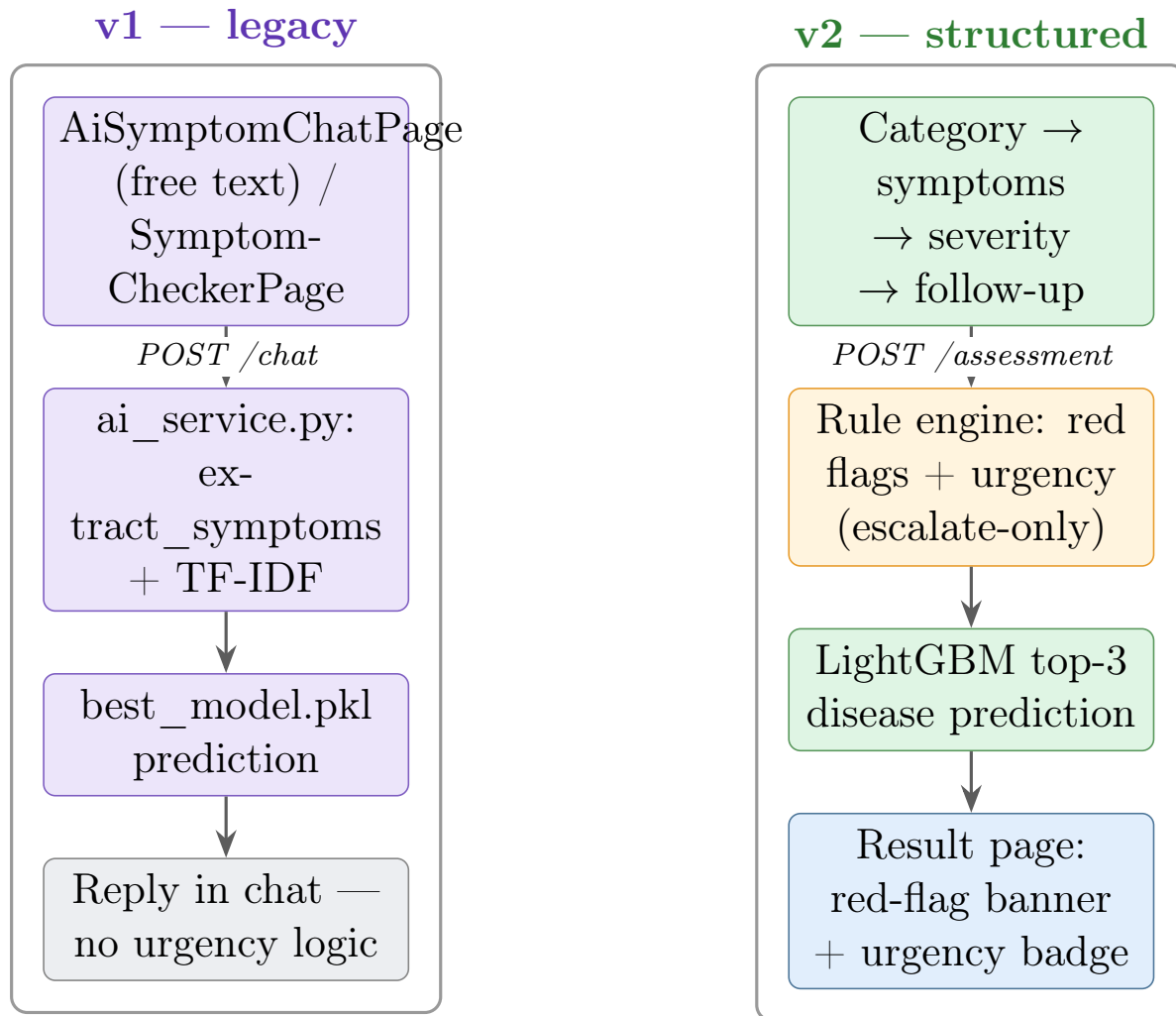


Figure 8.6: The two coexisting symptom-checker flows. v1 has no safety layer; v2 runs the ML prediction and the rule engine independently and reflects the more severe of the two.

8.4.1 The Safety Rule Engine

Two pure-function rule modules sit between the raw assessment input and the final result, and — per an explicit design rule in the code — **the rule engine can only escalate urgency, never suppress it:**

- **red_flag_rules** — most red-flag symptoms trigger at severity ≥ 2 ; syncope and loss-of-consciousness variants trigger at any severity ≥ 1 , reflecting their inherent urgency.
- **urgency_rules** — combines red flags with a vitals-derived floor into one of four levels: *critical* (any red flag at severity ≥ 3 , or critical vitals), *high* (any red flag, or high vitals), *moderate* (≥ 2 moderate-severity symptoms, or known hypertension with elevated vitals), or *low*.
- **bp_rules** — the single shared source of BP/SpO2/HR thresholds, written to be the one place these numbers live rather than duplicated between the symptom checker and the BP feature's own alerting.

When an assessment is launched from a concerning BP reading specifically, and the result comes back moderate or high (not yet critical), the response additionally recommends resting five minutes and re-measuring rather than escalating immediately — the concrete implementation of the “re-measure on a borderline high reading, notify immediately on a confirmed critical one” policy.

8.5 Application Screenshots

The screens below are grouped as complete user flows, captured with the app set to **Arabic**, so the sequence itself is direct proof that the interface runs fully and naturally right-to-left — not just isolated screens.

Flow 1 — Guided blood-pressure measurement

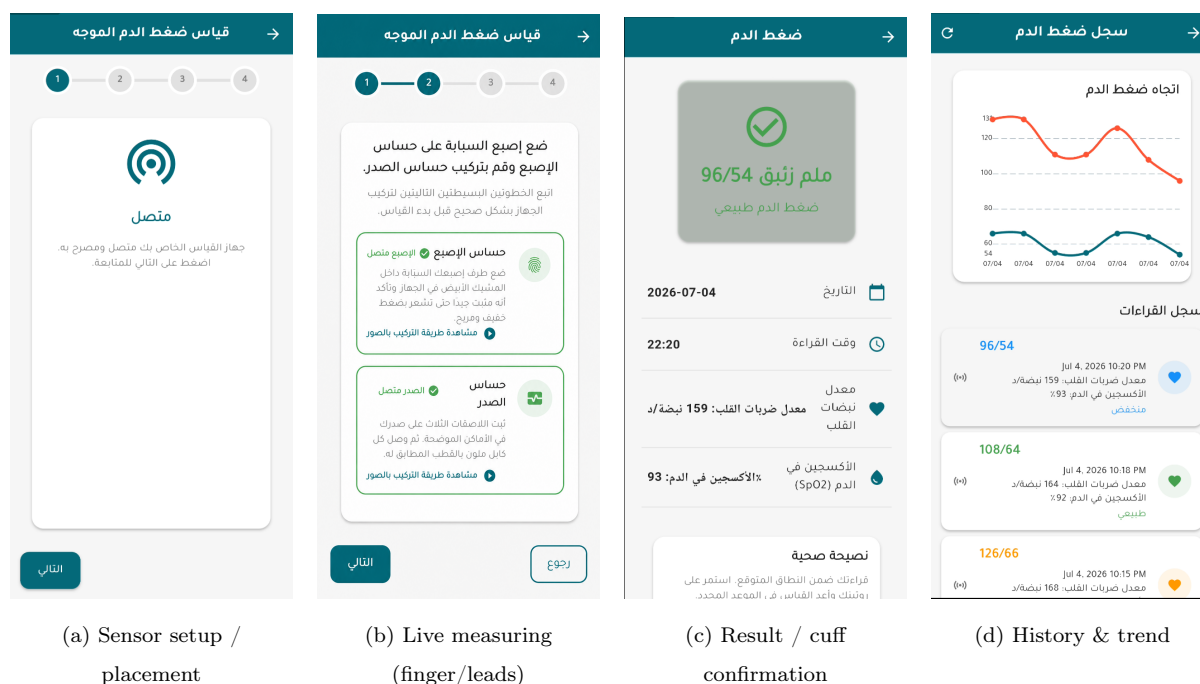


Figure 8.7: Guided blood-pressure measurement, from sensor setup to saved history (Arabic UI).

Flow 2 — Medication, from setup to alarm

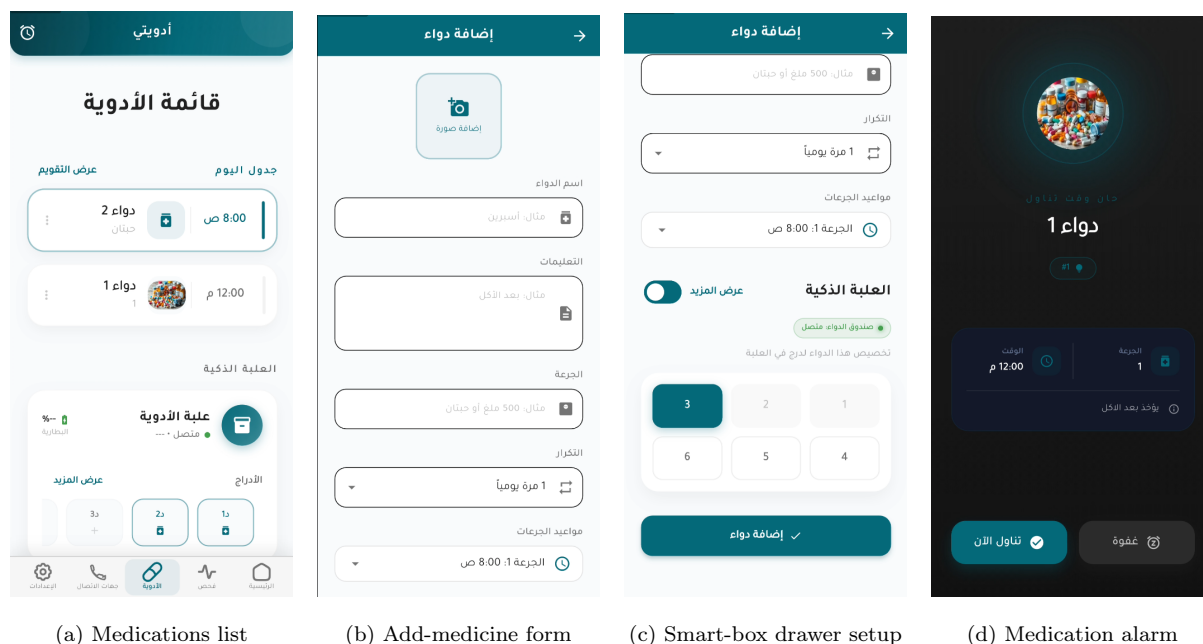


Figure 8.8: Medication flow: list, add-medicine form, smart-box setup, and the resulting alarm (Arabic UI).

Flow 3 — Symptom triage

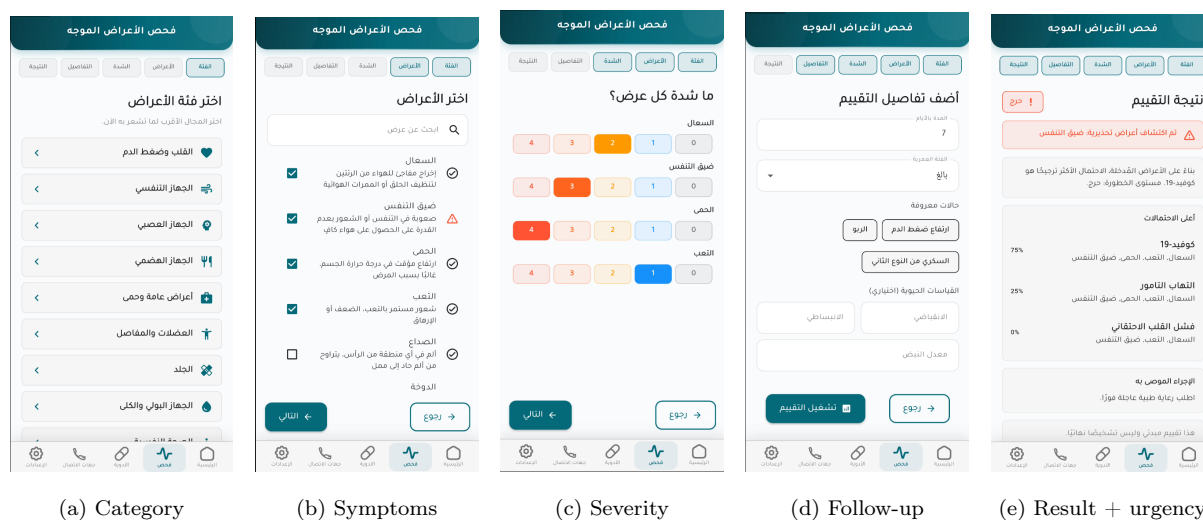


Figure 8.9: Symptom-triage wizard: category, symptoms, severity, follow-up, and the urgency-graded result (Arabic UI).

Flow 4 — Communication

Reaching a linked patient or caregiver directly from the app, including the dedicated emergency-call entry point (Figure 8.10).

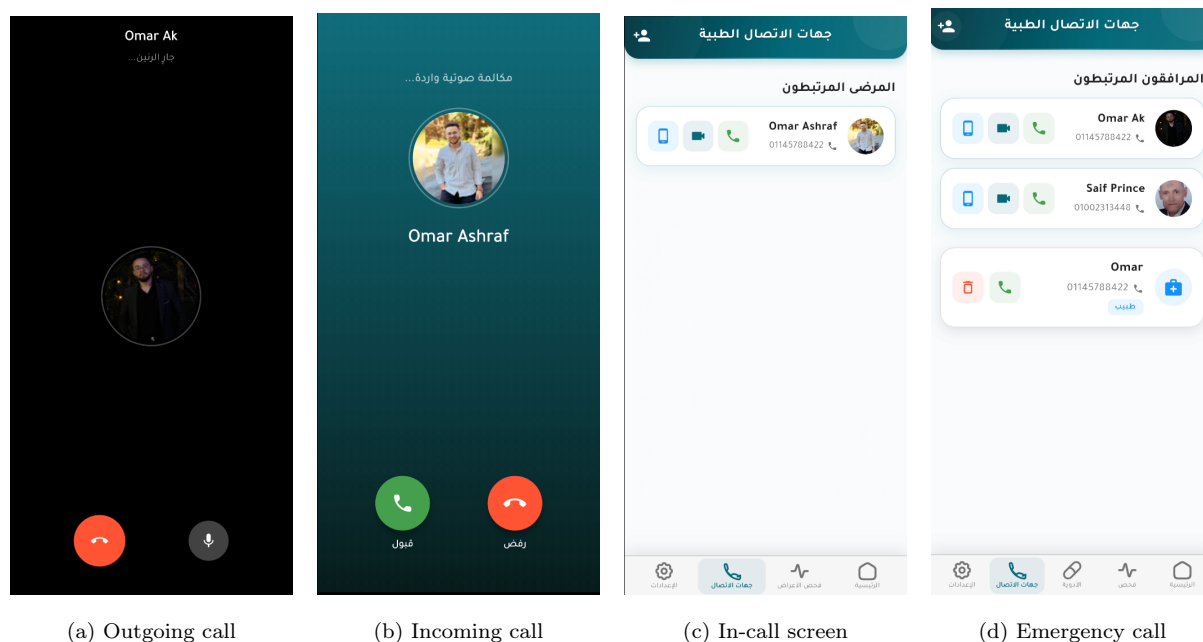


Figure 8.10: WebRTC audio/video calling: outgoing, incoming, active call, and the emergency-call screen (Arabic UI).

Core screens

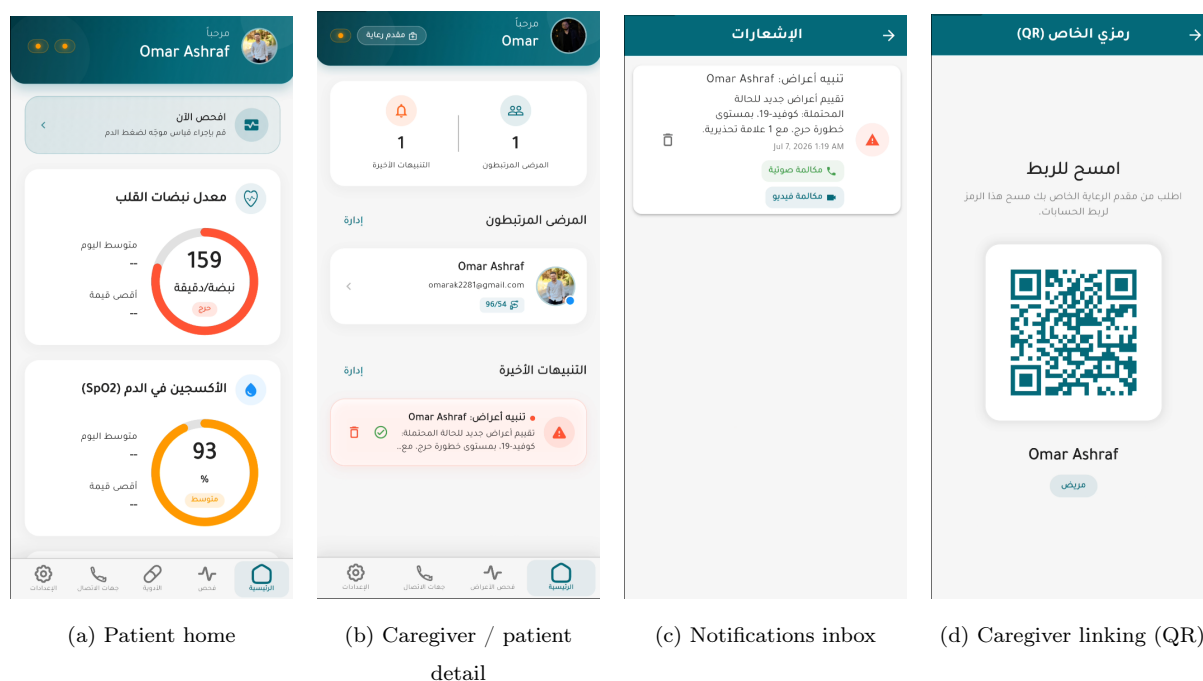


Figure 8.11: Patient home, caregiver patient detail, notifications inbox, and QR-based caregiver linking (Arabic UI).

Chapter 9

Impact

9.1 Target Audience

The primary users are patients managing chronic cardiovascular conditions at home and the family members or informal carers who monitor them remotely. The system deliberately does *not* model a clinical-staff or clinic-admin role: `UserRole` is a two-value enum (`patient`, `caregiver`), and no schema, endpoint, or screen anywhere in the code-base suggests a broader role model was ever built or even partially scaffolded. Its scope is patient-and-family, not patient-and-provider, and that scoping decision shapes every screen in the app: there is no clinician dashboard to design around, no multi-tenant clinic account to secure, and no provider-facing reporting format to satisfy — the entire interface can be optimized for exactly two perspectives on the same underlying data. Three concrete groups of people benefit directly from that focus, and the specific design choices throughout the app map onto their real daily difficulties rather than a generic “health app” feature list:

- **Elderly patients managing hypertension.** Older adults are both the demographic most likely to live with hypertension and the demographic least well served by an app designed primarily for younger, tech-fluent users. Health Mate answers this directly: a large, simple UI; a step-by-step guided measurement that removes any guesswork about how to attach and use the sensor correctly; scheduled reminders so the patient never has to hold a measurement schedule in their own head; and full Arabic support with right-to-left layout, so language is never the reason a patient cannot use the app. A normal cuff measurement actually asks an older patient to get three separate things right — correct cuff placement, sitting still for an accurate reading, and then remembering to write the number down somewhere — and any one of the three going wrong can mean the reading is skipped entirely. The guided flow collapses all three into a single prompted routine, so there is no step left for the patient to get wrong on their own.
- **Any hypertension patient who needs to track their condition.** For the broader population managing high blood pressure, day to day, the value is continuity rather than any single reading. Instead of isolated clinic snapshots taken weeks or months apart, the patient accumulates a real, continuous history that they and their caregiver can both see, and — most importantly for actual safety — a critical reading is escalated to a linked caregiver automatically and immediately, rather than depending on the patient themselves noticing the danger, judging its severity correctly, and then finding the presence of mind to raise the alarm while acutely unwell. The per-patient calibration layer means the numbers this patient sees become progressively more trust-

worthy specifically for their own physiology over time, rather than only being accurate on average across the population the underlying model was trained on.

- **Patients with dementia or Alzheimer’s who need medication support.** Memory-impairment patients face a distinct problem that the blood-pressure features do not address at all: not *measuring* anything, but *remembering which medication to take, and when*. The smart medicine box speaks directly to this gap. Rather than relying on the patient to hold a schedule in memory, it lights the LED for exactly the drawer that is due right now (and sounds a buzzer), turning an abstract fact — “it is 8 pm, time for the evening dose” — into an unambiguous physical pointer at one specific compartment, reinforced at the same moment by a phone notification. For a patient who cannot reliably retain a schedule, a device that points directly at the answer is materially more usable than one that only reminds them a decision is due and leaves them to work out which pill that means on their own. A linked caregiver sees the resulting adherence log, so a missed dose is visible to someone who is in a position to follow up, rather than silently going unnoticed.

9.2 Problems Solved

The table below restates the same value proposition as a direct map from a concrete, real-world problem to the specific mechanism in the running system that addresses it — deliberately including a few smaller but genuinely useful mechanisms (offline caching, the emergency-call screen) that are easy to overlook next to the two headline AI features, but that materially affect whether the app is actually usable in the situations patients and caregivers run into day to day.

Table 9.1: Real-world problems and the Health Mate mechanism that solves each.

Problem	How Health Mate addresses it
Patients skip or forget blood-pressure measurement	Up to three scheduled daily reminders, paired with a guided, low-friction sensor flow that requires no manual cuff technique at all
A dangerous reading goes unnoticed until it becomes an emergency	Automatic risk-tier classification with a rate-limited emergency push notification sent to every linked caregiver the moment a high or critical reading is recorded
Caregivers have no visibility into a patient's condition without asking	A live, linked view of the patient's vitals, calibration status, and medication adherence, requiring no ongoing action from the patient after the initial link is made
A cuff reading is intimidating or physically hard for an elderly patient to self-administer correctly	An 8-second cuffless sensor capture, walked through step by step, with live finger/lead feedback so a bad measurement attempt is caught and corrected before a reading is even taken
A model trained on a general population is inaccurate for one specific individual	A per-patient calibration layer that adapts to occasional reference cuff readings and automatically detects when its own fit has drifted out of date
Dementia/Alzheimer's patients forget which medication to take, and when	The smart medicine box lights a per-drawer LED (with a buzzer) pointing at the exact compartment due right now, reinforced by a phone alarm
There is no record of whether a medication dose was actually taken	An adherence log with optional photo proof, visible to the linked caregiver
Reaching help quickly during a frightening reading or symptom episode	In-app WebRTC audio/video calling, including a dedicated emergency-call screen between patient and caregiver
Symptoms that may be urgent are self-judged by an unwell, possibly panicking patient	A structured symptom-triage wizard backed by a rule engine that can escalate urgency independently of the ML prediction and can never suppress a red flag, regardless of what the model predicts
Arabic-speaking users are treated as a secondary market by most health apps	Full English/Arabic parity at the translation-key level, right-to-left layout, and translated reminders and alerts, not just translated static text
Losing access to recent data the moment connectivity drops	Offline-first caching of the last known reading and history, with local reminders that continue to fire even with no network connection at all

9.3 Innovative Aspects of the Design

- A genuinely **fault-tolerant, resumable model-training pipeline**, built out of practical necessity for training on a quota-limited shared GPU platform, in which every stage checkpoints and one model's failure never takes down the run.
- A **calibration subsystem** that adapts a population-trained model to an individual patient without retraining it, upgrading its own fit quality automatically as more reference data accumulates.
- A **safety rule engine** architecturally guaranteed to only escalate, never suppress, urgency relative to the ML model's own output.
- A **single shared feature-engineering implementation** used by both training and production inference on both AI subsystems, eliminating train/inference drift by construction.
- A **clean separation of clinical logic from hardware**: the devices are pure collectors/actuators, and every safety-relevant decision runs in one auditable backend.

9.4 Reliability

Reliability was treated as a first-class concern in the parts of the system that matter most. The BP model service and the smart-box hardware client both fail soft rather than raising on a missing model or unreachable device. The rule engine, calibration service, and drift detector each carry dedicated unit tests — the backend's test investment is concentrated on exactly the newest, safety-critical code (a rule-engine test suite, BP endpoint and drift tests, assessment and LLM-fallback tests, a Redis chat-session test, and an automated Arabic-encoding audit), rather than spread evenly.

Where reliability gaps remain, this documentation states them directly rather than implying a finished, hardened system:

- **Session/token revocation is not implemented** — logout and refresh do not invalidate anything server-side; this is a standard stateless JWT tradeoff, stated in the code's own docstring.
- **The caregiver-linking QR flow is unencrypted** — the QR code carries a plain user UUID with no signing or expiry.
- **The generic sensor endpoints always return simulated data**, regardless of IoT mode, which could be mistaken for a live feed if read from the API surface alone.

9.5 Added Value

For a patient, the system converts BP monitoring from an easy-to-skip manual task into a scheduled, guided routine with a direct line to a caregiver in an emergency. For a

caregiver, it converts remote monitoring from “wait to be told” into a live, linked view with automatic alerting. The bilingual, RTL-aware design makes both experiences equally first-class in Arabic and English, which materially broadens the population the system can actually serve.

Chapter 10

Conclusions

Health Mate is a working, end-to-end remote-monitoring system spanning custom sensor hardware, two trained AI models, a FastAPI backend that centralizes every safety decision, and a bilingual Flutter app serving both patients and caregivers from shared infrastructure. The blood-pressure model clears the clinical standard for diastolic pressure but not yet for systolic; the symptom-checker model reaches strong top-3 accuracy overall but is weakest precisely on the cardiac subtypes most relevant to the project's focus. Both results are reported as measured, not adjusted for presentation.

The project's biggest lesson, encountered independently in both AI subsystems, was that a more sophisticated architecture does not automatically outperform a simpler one on a given dataset, and that discovering this requires actually running the simpler baseline rather than assuming the more complex model is better by construction. A second, quieter lesson concerns engineering discipline: sharing one preprocessing and feature-engineering implementation between training and inference, failing soft on every best-effort integration, and keeping the safety layer as pure, testable functions independent of the ML model together produced a system whose behavior can be reasoned about and trusted at its most safety-critical points.

Above all, the project was documented honestly. Every headline number is the one the code actually produces, every architectural gap is named, and every inconclusive result is left inconclusive rather than rounded up. This is the posture a real medical product demands, and adopting it at the graduation-project stage is itself part of the contribution.

Chapter 11

Future Directions

- **Validate the BP model against real hardware.** Collect MAX30102/AD8232 recordings alongside reference cuff readings and measure the model’s actual MAE on that data — the single largest open validation question, and one that lies in the absence of a measurement rather than in the code.
- **Re-run the calibration effectiveness study** using genuinely time-separated reference readings that mirror how the deployed service accumulates samples, rather than adjacent overlapping windows from one recording.
- **Investigate the complexity paradox.** Determine why additional architectural sophistication underperformed a simpler baseline in both subsystems — candidates include insufficient data diversity for attention to earn its parameters, and an inductive-bias mismatch for fixed-length windows.
- **Strengthen cardiac-subtype discrimination.** Collect real (non-synthetic) patient assessment data and apply the same targeted differentiation to overlapping cardiac classes that was applied to the respiratory and neurological triads.
- **Close the backend security gaps** — server-side session/token revocation and cryptographic protection (signing + expiry) for the caregiver-linking flow — before any production deployment.
- **Harden device provisioning** — replace the hardcoded firmware device token with a per-device provisioned secret.
- **Decide the long-term status of the legacy v1 symptom checker** now that a safer, structured v2 flow exists alongside it, and consider the taxonomy’s own anticipated “Phase 6” move to a Postgres-backed repository.

11.1 Hardware Sensor Upgrade Path

The current prototype uses two low-cost breakout modules — a MAX30102 PPG sensor and an AD8232 single-lead ECG front-end — chosen for availability and price during development. Both are functional but are consumer/fitness-grade parts, and a clear, researched upgrade path exists toward medical-grade signal acquisition, which is the most direct lever on the SBP accuracy gap identified in Chapter 7:

- **Combine PPG and ECG into one medical-grade module.** The MAX86150 [13] integrates a PPG sensor *and* a single-lead ECG channel in one part with a shared,

synchronized analog front end. This directly replaces *both* current sensors, removes the cross-module timing uncertainty that affects the pulse-transit-time feature, and has been reported in cuffless-BP evaluations at a mean error on the order of ± 5.3 mmHg systolic and ± 2.8 mmHg diastolic — squarely in the range that would move the systolic result toward the AAMI bar.

- **Upgrade the ECG front end.** If PPG and ECG are kept separate, the **MAX30003** [14] is a medical-grade biopotential AFE with built-in EMI filtering, DC lead-off detection, internal lead biasing, and ultra-low power draw — a substantial signal-quality improvement over the AD8232’s motion-artifact-tolerant but fitness-oriented design.
- **Adopt a validated reference design.** Vendor cuffless-BP reference designs such as the **MAXREFDES220** [15] are documented to meet Class-II regulatory accuracy limits under the **ISO 81060-2** [9] clinical-validation standard, and could serve as a hardware baseline to validate the trained model against, closing the real-hardware validation gap named above.

Pairing any of these upgrades with the real-hardware data-collection step above would let the model be retrained and re-validated on higher-fidelity signals from the actual target population, which is the single change most likely to turn the systolic result from an indicative trend into a diagnostic-grade reading.

Bibliography

- [1] A. E. W. Johnson, T. J. Pollard, L. Shen, et al., “MIMIC-III, a freely accessible critical care database,” *Scientific Data*, vol. 3, art. 160035, 2016. doi:10.1038/sdata.2016.35.
- [2] M. Kachuee, M. M. Kiani, H. Mohammadzade, and M. Shabany, “Cuff-less high-accuracy calibration-free blood pressure estimation using pulse transit time,” in *Proc. IEEE Int. Symp. Circuits and Systems (ISCAS)*, 2015, pp. 1006–1009.
- [3] F. Schrumpf, P. Frenzel, C. Aust, G. Osterhoff, and M. Fuchs, “Assessment of deep learning based blood pressure prediction from PPG and rPPG signals,” in *Proc. IEEE/CVF CVPR Workshops*, 2021. arXiv:2104.09313.
- [4] S. Baker, W. Xiang, and I. Atkinson, “A deep learning approach to predict blood pressure from PPG signals,” *Computers in Biology and Medicine*, 2021. arXiv:2108.00099.
- [5] G. Ke, Q. Meng, T. Finley, et al., “LightGBM: A highly efficient gradient boosting decision tree,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [6] A. Vaswani, N. Shazeer, N. Parmar, et al., “Attention is all you need,” in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [7] J. M. Bland and D. G. Altman, “Statistical methods for assessing agreement between two methods of clinical measurement,” *The Lancet*, vol. 327, no. 8476, pp. 307–310, 1986.
- [8] E. O’Brien, J. Petrie, W. Littler, et al., “The British Hypertension Society protocol for the evaluation of automated and semi-automated blood pressure measuring devices,” *Journal of Hypertension*, vol. 11 (suppl. 2), pp. S43–S62, 1993.
- [9] Association for the Advancement of Medical Instrumentation, *ANSI/AAMI/ISO 81060-2: Non-invasive Sphygmomanometers — Clinical Investigation of Automated Measurement Type*.
- [10] S. Bai, J. Z. Kolter, and V. Koltun, “An empirical evaluation of generic convolutional and recurrent networks for sequence modeling,” arXiv:1803.01271, 2018.
- [11] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [12] J. L. Ba, J. R. Kiros, and G. E. Hinton, “Layer normalization,” arXiv:1607.06450, 2016.
- [13] Maxim Integrated (Analog Devices), *MAX86150 Integrated Photoplethysmogram and Electrocardiogram Bio-Sensor Module for Mobile Health*, Datasheet.

- [14] Maxim Integrated (Analog Devices), *MAX30003 Ultra-Low Power, Single-Channel Integrated Biopotential (ECG, R-to-R Detection) Analog Front End*, Datasheet.
- [15] Maxim Integrated (Analog Devices), *MAXREFDES220# — Cuffless Blood-Pressure Measurement Reference Design*, meeting Class-II regulatory accuracy limits (ISO 81060-2).

Appendix A

API Endpoint Reference

All endpoints are under the base path `/api/v1`. Two authentication schemes coexist: **Bearer JWT** for all human-user endpoints, and **device headers** (`X-Device-ID` + `X-Device-Token`) for the two BP hardware endpoints only.

Table A.1: REST API surface, grouped by router.

Endpoint	Method	Purpose
<i>Authentication (/auth)</i>		
<code>/auth/register</code>	POST	Create a patient or caregiver account
<code>/auth/login</code>	POST	Email/password login, returns access + refresh tokens
<code>/auth/refresh</code>	POST	Exchange a refresh token for a new pair
<code>/auth/social</code>	POST	Firebase-based login/signup
<code>/auth/verify-email</code>	POST	Mark email verified
<code>/auth/logout</code>	POST	Returns success; no server-side invalidation (known gap)
<i>Users (/users)</i>		
<code>/users/me</code>	GET/PUT/DELETE	User profile management
<code>/users/me/fcm-token</code>	PUT	Register/update push token
<code>/users/me/password</code>	PUT	Change password
<code>/users/linked</code>	GET	Linked caregivers or patients
<code>/users/link/{id}</code>	POST	Link with another user by ID
<code>/users/linked/{id}</code>	DELETE	Deactivate a link
<i>Vitals / Blood pressure (/vitals)</i>		
<code>/vitals/bp</code>	POST	Manual BP entry
<code>/vitals/bp/submit</code>	POST	Device submits 800-sample window; model + calibration
<code>/vitals/bp/complete</code>	POST	Patient supplies cuff reading, records calibration sample
<code>/vitals/bp/heartbeat</code>	POST	Device reports finger/lead status
<code>/vitals/bp/history</code>	GET	Paginated reading history

Table A.1 continued

Endpoint	Method	Purpose
/vitals/bp/stats	GET	Aggregate statistics for charts
/vitals/patient/{id}/*	GET	Same views for a linked patient (caregiver)
Medications, reminders, notifications		
/medications	CRUD	Catalog, schedule, smart-box assignment, adherence
/bp_reminders/schedule-daily	POST	Derive up to 3 daily BP reminder times
/notifications	GET/PUT/DELETE	Read-count, mark-read, delete
/contacts	CRUD	Medical contacts
Calls (/calls)		
/calls	POST	Start a call
/calls/{id}/offer,accept,reject,offer,end	POST	WebRTC session lifecycle
IoT / Hardware		
/iot/sensors/status, /data	GET	Simulated sensor telemetry (mock, always)
/iot/medicine-box/*	GET/POST	Real ESP32 medicine-box state and control
/hardware/status	GET	ESP32 reachability ping
AI / Symptom checker (/ai)		
/ai/symptom-checker, /chat	POST	v1 legacy classifier + free-text chat
/ai/categories, /taxonomy/symptoms	GET	v2 taxonomy
/ai/assessment	POST	v2 structured assessment
/ai/bp-triage	POST	v2 assessment + borderline-high re-measure override
/ai/chat/from-assessment	POST	Seed a chat session with assessment context
/ai/assessment/notify-caregiver	POST	Explicit caregiver notification trigger
/ai/assessment/rollout-metrics	GET	Live counters: count, red-flag rate, latency
Upload / Realtime		
/upload/*	POST/DELETE	Cloudinary-backed image upload
/ws, /socket.io	WS	JWT-authenticated WebRTC signaling events

Appendix B

Deployment and Networking

The system runs via a single `docker-compose.yml` (PostgreSQL, Redis, and the FastAPI API container) on one host — a development/demo topology with no staging/production split, no reverse proxy/TLS, and no orchestration. The `Symptom-Checker/` and `Predict-ABP/` directories are bind-mounted into the API container rather than copied at build time, so model artifacts can be swapped without rebuilding the image.

Networking between the phone, backend, and both ESP devices depends on all being on the same WiFi network, with IP addresses hardcoded in four separate files that must be updated by hand when the network changes: the frontend API base URL and ESP32 constant, the backend’s ESP32 IP setting, and the BP firmware’s server URL. This is a real, hand-maintained coupling and a documented operational step, not an oversight.

Appendix C

Documentation Method and Honesty Statement

Every factual claim in this document was produced by reading primary sources directly, in priority order: (1) source code — Python backend modules, Dart widgets and providers, Arduino/ESP firmware, SQLAlchemy models, Pydantic schemas, and Alembic migrations; (2) generated artifacts — model metadata JSON, evaluation reports, result CSVs, and the training notebook’s own cell outputs, treated as primary evidence because they are produced by running code, not by describing intent; and (3) direct cross-verification, where a claim that could be checked on two sides (for example, whether the linking QR code encodes an encrypted payload) was read on both the backend and the Flutter side rather than trusting one comment.

Several claims from earlier project materials were corrected during this review: the blood-pressure model is a 1D-CNN, not an LSTM; the production symptom-checker model is LightGBM over structured features, with an older TF-IDF pipeline still running in parallel rather than in its place; the previously-quoted “90.3% accuracy” is disowned by the current model’s own metadata as measured on a non-representative synthetic method; the caregiver-linking QR code is not encrypted; and the smart medicine box drives only LEDs and a buzzer, with no motor or dispensing mechanism of any kind. Reporting these corrections, rather than repeating the older claims, is a deliberate part of this document’s method.

Appendix D

Code Snapshots

These short snapshots are copied from the real project source files and trimmed to show only the implementation decisions worth highlighting. They are placed last so the main documentation remains readable while still giving reviewers concrete evidence from the codebase.

D.1 Hardware BP Submission Gate

```
1 # Run prediction model (or Keras engine)
2 from app.services.abp_prediction_service import get_abp_service
3 abp_service = get_abp_service()
4 prediction = abp_service.predict_bp(payload.ppg_signal, payload.ecg_signal)
5
6 if prediction.get("prediction_status") == "model_not_ready":
7     raise HTTPException(
8         status_code=status.HTTP_503_SERVICE_UNAVAILABLE,
9         detail="model_not_ready"
10    )
11
12 # SpO2 is only trusted inside the physiologically plausible 70-100% band;
13 # anything else is a sensor-quality failure and stored as NULL, never as a
14 # bad number.
15 spo2_value = (
16     payload.spo2
17     if payload.spo2 is not None and 70 <= payload.spo2 <= 100
18     else None
19 )
```

Listing D.1: Backend snapshot from `app/api/v1/vitals.py`: hardware readings are model-checked, validated, and rejected safely before persistence.

D.2 Calibration-Aware Measurement State

```
1 # Get calibration factors if available
2 cal_service = get_calibration_service()
3 cal_res = await cal_service.apply_calibration(
4     db, device.patient_id, prediction["systolic"], prediction["diastolic"]
5 )
6
7 calibration_status = cal_res["calibration_status"]
8 needs_cuff_confirmation = calibration_status in {
9     "not_calibrated",
10    "weak",
11    "recalibration_due",
12 }
13 systolic = None if needs_cuff_confirmation else int(round(cal_res["systolic"]))
14 diastolic = None if needs_cuff_confirmation else int(round(cal_res["diastolic"]))
15 measurement_status = (
16     MeasurementStatus.COMPLETED_PENDING_BP
```

```

17     if needs_cuff_confirmation
18     else MeasurementStatus.COMPLETED
19 )

```

Listing D.2: Backend snapshot from `app/api/v1/vitals.py`: untrusted calibrated readings are kept pending until cuff confirmation.

D.3 ABP Model Inference Core

```

1  # 1. Preprocessing (filtering & z-score normalizing)
2  ppg_f = bandpass(ppg, 0.5, 8.0, FS)
3  ecg_f = bandpass(ecg, 0.5, 40.0, FS)
4
5  ppg_n = normalize_wave(ppg_f)
6  ecg_n = normalize_wave(ecg_f)
7
8  # Form waveform input array shape (1, 800, 2)
9  wave_input = np.stack([ppg_n, ecg_n], axis=-1)[np.newaxis].astype(np.float32)
10
11 # 2. Extract features shape (1, 21)
12 feats = extract_features(ppg_n, ecg_n)
13 feat_sc = self.sc_x.transform(feats.reshape(1, -1)).astype(np.float32)
14
15 # 3. Model run
16 pred_sc = self.model({'waveform_input': wave_input, 'feature_input': feat_sc}, training=False).numpy()
17 pred = self.sc_y.inverse_transform(pred_sc)[0]
18
19 sbp = float(np.clip(float(pred[0]), 50.0, 300.0))
20 dbp = float(np.clip(float(pred[1]), 30.0, 200.0))

```

Listing D.3: Backend snapshot from `app/services/abp_prediction_service.py`: waveform preprocessing, feature scaling, Keras inference, and physiological clamping.

D.4 Symptom-Triage Safety Path

```

1  _resolve_red_flags(assessment, taxonomy)
2  red_flags = evaluate_red_flags(assessment.symptoms)
3  rule_urgency = determine_urgency(assessment, red_flags)
4
5  try:
6      predictions = classifier.predict_top_k(assessment, k=3)
7  except Exception: # noqa: BLE001
8      logger.exception("Disease classifier failed - degrading to rule-engine-only result")
9      predictions = []
10
11 ml_urgency = rule_urgency
12 top_predictions: list[TopPrediction] = []
13 selected_ids = {s.id for s in assessment.symptoms}
14 for pred in predictions:
15     disease = taxonomy.get_disease(pred.disease_id)
16     if disease is None:
17         continue
18     why = sorted(selected_ids & set(disease.related_symptom_ids))
19     if disease.category_id == "heart_bp" and classify_bp_vitals(assessment.vitals) in ("high",
20         "critical"):
21         why.append("elevated_bp")

```

Listing D.4: Backend snapshot from `app/application/use_cases/run_assessment.py`: rule-engine safety still runs even if the classifier fails.

D.5 Escalate-Only Urgency Rule

This is the single most repeated safety claim in this document, so it is worth showing the exact function that enforces it. Note there is no code path in which `determine_urgency` can be told to *lower* an already-detected red flag or vitals floor — each branch can only return an equal or higher tier than the one before it.

```

1 def determine_urgency(assessment, red_flags):
2     vitals_urgency = classify_bp_vitals(assessment.vitals)
3
4     if any(rf.severity >= CRITICAL_RED_FLAG_SEVERITY_THRESHOLD
5           for rf in red_flags) or vitals_urgency == "critical":
6         return "critical"
7
8     if red_flags or vitals_urgency == "high":
9         return "high"
10
11    moderate_count = sum(1 for s in assessment.symptoms
12                        if s.severity >= MODERATE_SYMPTOM_SEVERITY_THRESHOLD)
13    has_hypertension = "hypertension" in {c.lower() for c in assessment.known_conditions}
14    if moderate_count >= MODERATE_SYMPTOM_COUNT_THRESHOLD or (
15        has_hypertension and is_bp_elevated(assessment.vitals)
16    ):
17        return "moderate"
18
19    return "low"

```

Listing D.5: Domain snapshot from `app/domain/rules_engine/urgency_rules.py`: red flags and vitals set an urgency floor the ML classifier can never override downward.

D.6 Primary-Caregiver Database Constraint

```

1 __table_args__ = (
2     Index(
3         "uq_patient_active_primary_caregiver",
4         "patient_id",
5         postgresql_where="is_primary = true AND is_active = true",
6         unique=True
7     ),
8 )

```

Listing D.6: Schema snapshot from `app/models/patient_caregiver_link.py`: PostgreSQL enforces one active primary caregiver per patient.

D.7 Firmware Signal-Presence Gating

The one hardware-side snippet worth including: the BP sensing unit refuses to buffer a sample at all unless both signals are physically present, which is what keeps a bad measurement from ever reaching the model in the first place.

```

1 bool checkGating() {
2     // 1. Leads Status: AD8232 leads must be connected
3     bool leadsOk = (digitalRead(LO_PLUS) == LOW && digitalRead(LO_MINUS) == LOW);
4     if (!leadsOk) {
5         Serial.println("GATING_ERROR:ECG_LEADS_DETECTED_DISCONNECTED");
6         return false;
7     }
8 }

```

```
9 // 2. Finger Detection: PPG IR must detect a finger (> 50,000 counts)
10 ppg.check();
11 long ir = ppg.getIR();
12 if (ir < 50000) {
13     Serial.println("GATING_ERROR:PPG_NO_FINGER_DETECTED");
14     return false;
15 }
16
17 return true;
18 }
```

Listing D.7: Firmware snapshot from `bp-hardware-code/bp-hardware-code.ino`: the ESP8266 gates every sample on ECG lead contact and PPG finger presence before it is ever buffered.

D.8 Flutter Symptom-Checker API Call

```
1 Future<Map<String, dynamic>> runAssessment(
2     Map<String, dynamic> body, String lang) async {
3     final response = await _dioClient.dio.post(
4         ApiConstants.aiAssessment,
5         queryParameters: {'lang': lang},
6         data: body,
7     );
8     return Map<String, dynamic>.from(response.data as Map);
9 }
```

Listing D.8: Flutter snapshot from `symptom_checker_remote_datasource.dart`: the mobile app sends structured assessment input directly to the v2 backend endpoint.